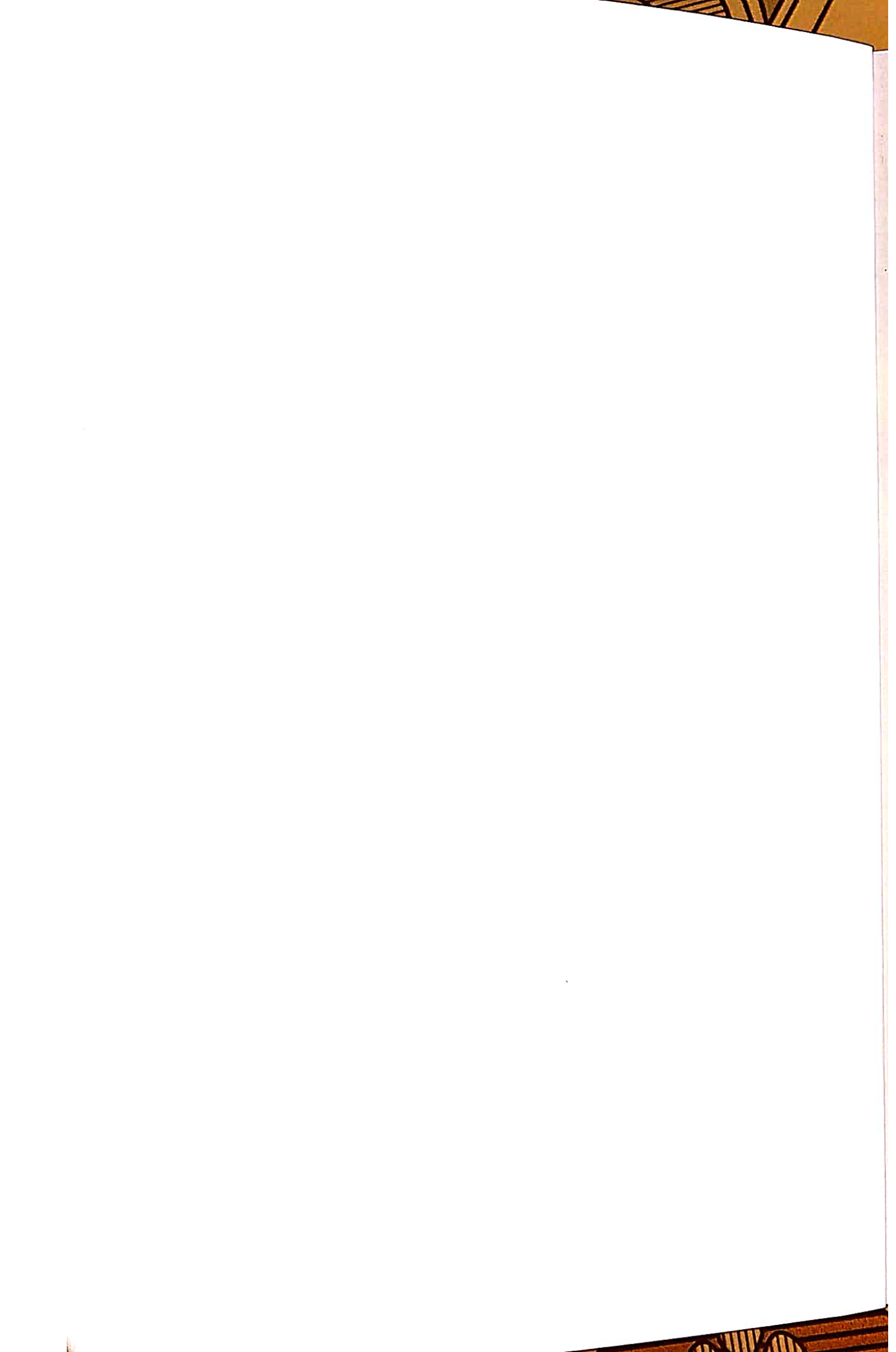




SUBJECT CODE : 410241

Choice Based Credit System

SAVITRIBAI PHULE PUNE UNIVERSITY - 2019 SYLLABUS

B.E. (Computer) Semester - VII

DESIGN AND ANALYSIS OF ALGORITHMS

(For END SEM Exam - 70 Marks)

Mrs. Anuradha A. Puntambekar

M.E. (Computer)

Formerly Assistant Professor in
P.E.S. Modern College of Engineering,
Pune.

FEATURES

- ☒ Written by Popular Authors of Text Books of Technical Publications
 - ☒ Covers Entire Syllabus
 - ☒ Question - Answer Format
 - ☒ Exact Answers and Solutions
- ☒ Chapterwise Solved SPPU Questions Dec. 2006 to March 2020
 - ☒ Solved Model Question Paper (As Per 2019 Pattern)

SOLVED SPPU QUESTION PAPERS

- Dec. - 2016 • May - 2017 • Dec. - 2017 • May - 2018
- Dec. - 2018 • May - 2019 • Dec. - 2019 • March - 2020

DECODE[®]

A Guide For Engineering Students



A Guide For Engineering Students

DESIGN AND ANALYSIS OF ALGORITHMS

(For END SEM Exam - 70 Marks)

SUBJECT CODE : 410241

B.E. (Computer Engineering) Semester - VII

© Copyright with Technical Publications

All publishing rights (printed and ebook version) reserved with Technical Publications.

No part of this book should be reproduced in any form, Electronic, Mechanical, Photocopy or any information storage and retrieval system without prior permission in writing, from Technical Publications, Pune.

Published by :



Amit Residency, Office No.1, 412, Shaniwar Peth,
Pune - 411030, M.S. INDIA Ph.: +91-020-24495496/97

Email : info@technicalpublications.in

Website : www.technicalpublications.in

Printer :

Yogiraj Printers & Binders, Sr.No. 10/1A, Ghule Industrial Estate, Nanded Village Road,
Tal. - Haveli, Dist. - Pune - 411041.

ISBN 978-93-5585-239-7



9 789355 852397

SPPU 19

9789355852397 [1]

(ii)

SYLLABUS

Design and Analysis of Algorithms - (410241)

Credit	Examination Scheme :
03	End-Sem (Paper) : 70 Marks

Unit III Greedy And Dynamic Programming algorithmic Strategy

Greedy strategy : Principle, control abstraction, time analysis of control abstraction, knapsack problem, scheduling algorithms-Job scheduling and activity selection problem.

Dynamic Programming : Principle, control abstraction, time analysis of control abstraction, binomial coefficients, OBST, 0/1 knapsack, Chain Matrix multiplication.

(Chapter - 3)

Unit IV Backtracking and Branch-n-Bound

Backtracking : Principle, control abstraction, time analysis of control abstraction, 8-queen problem, graph coloring problem, sum of subsets problem.

Branch-n-Bound : Principle, control abstraction, time analysis of control abstraction strategies - FIFO, LIFO and LC approaches, TSP, knapsack problem. (Chapter - 4)

Unit V Amortized Analysis

Amortized Analysis : Aggregate Analysis, Accounting Method, Potential Function method, Amortized analysis-binary counter, stack Time-Space tradeoff, Introduction to Tractable and Non-tractable Problems, Introduction to Randomized and Approximate algorithms, Embedded Algorithms : Embedded system scheduling (power optimized scheduling algorithm), sorting algorithm for embedded systems.

(Chapter - 5)

Unit VI Multithreaded and Distributed Algorithms

Multithreaded Algorithms - Introduction, Performance measures, Analyzing multithreaded algorithms, Parallel loops, Race conditions.

Problem Solving using Multithreaded Algorithms - Multithreaded matrix multiplication, Multithreaded merge sort.

Distributed Algorithms - Introduction, Distributed breadth first search, Distributed Minimum Spanning Tree.

String Matching - Introduction, The Naive string matching algorithm, The Rabin-Karp algorithm. (Chapter - 6)

TABLE OF CONTENTS

Unit III

Chapter - 3 Greedy and Dynamic Programming Algorithmic Strategy (3 - 1) to (3 - 34)

3.1 Introduction to Greedy Strategy	3 - 1
3.2 Knapsack Problem	3 - 2
3.3 Scheduling Algorithms - Job Scheduling	3 - 7
3.4 Activity Selection Problem	3 - 10
3.5 Introduction to Dynamic Programming	3 - 12
3.6 Binomial Coefficients	3 - 14
3.7 OBST	3 - 16
3.8 The 0/1 Knapsack	3 - 23
3.9 Chain Matrix Multiplication	3 - 29

Unit IV

Chapter - 4 Backtracking and Branch-n-Bound (4 - 1) to (4 - 43)

4.1 Principle	4 - 1
4.2 Control Abstraction and Time Analysis	4 - 4
4.3 Applications of Backtracking	4 - 6
4.4 The 8-Queen Problem	4 - 6
4.5 Graph Coloring Problem	4 - 15

4.6 Sum of Subsets Problem	4 - 20
4.7 Principle and Control Abstraction	4 - 23
4.8 Strategies- FIFO, LIFO and LC Approaches	4 - 25
4.9 Concept of Bounding.....	4 - 28
4.10 Traveling Sales Person Problem	4 - 28
4.11 Knapsack Problem.....	4 - 39
4.12 Difference between Backtracking and Branch and Bound Techniques	4 - 43

Unit V

Chapter - 5	Amortized Analysis	(5 - 1) to (5 - 14)
--------------------	---------------------------	----------------------------

5.1 Amortized Analysis.....	5 - 1
5.2 Time-Space Trade-off.....	5 - 6
5.3 Introduction to Tractable and Non-tractable Problems.....	5 - 7
5.4 Introduction to Randomized Algorithms.....	5 - 7
5.5 Introduction to Approximate Algorithms.....	5 - 9
5.6 Embedded Algorithms.....	5 - 10

Unit VI

Chapter - 6	Multithreaded and Distributed Algorithms	(6 - 1) to (6 - 18)
--------------------	---	----------------------------

6.1 Multithreaded Algorithms.....	6 - 1
6.2 Problem Solving using Multithreaded Algorithms	6 - 6
6.3 Distributed Algorithms.....	6 - 10
6.4 String Matching	6 - 11

Solved Model Question Paper	(M - 1) to (M - 2)
------------------------------------	---------------------------

Unit III

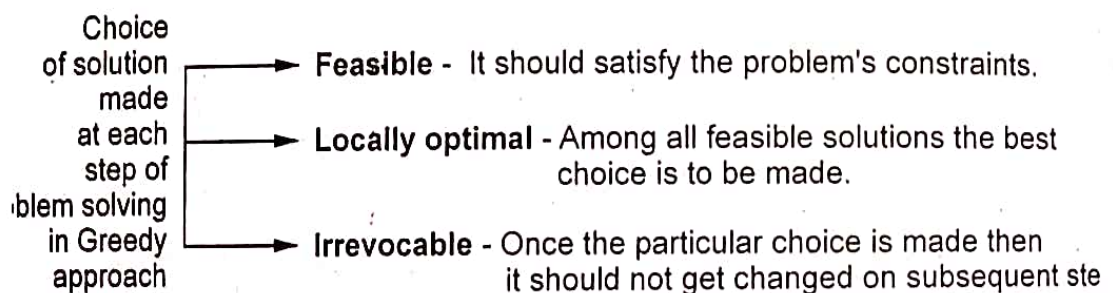
3

Greedy and Dynamic Programming Algorithmic Strategy

3.1 : Introduction to Greedy Strategy

Q.1 What is Greedy strategy ?

Ans. : • In Greedy technique, the solution is constructed through a sequence of steps, each expanding a partially constructed solution obtained so far, until a complete solution to the problem is reached. At each step the choice made should be,



• In short, while making a choice there should be a Greedy for the optimum solution.

Q.2 What are the applications of Greedy strategy ?

Ans. :

- 1) Knapsack problem
- 2) Job sequencing with deadlines
- 3) Minimum spanning trees
- 4) Single source shortest path algorithm.
- 5) Huffman's Tree

Q.3 What are the characteristics of greedy method ?

[SPPU : Dec.-06,07, May-07,09, Marks 8]

Ans. :

1. Greedy choice property : By this property, a globally optimal solution can be arrived at by making a locally optimal choice. That means, for finding the solution to the problem. We solve the subproblems, and whichever choice. We find best for that subproblem is considered. Then solve the subproblem is considered. Then solve the subproblem arising after the choice is made. This choice may depend upon the previously made choices but it does not depend on any future choice. Thus in greedy method, greedy choices are made one after the another, reducing each given problem instance to smaller one. The greedy choice property brings efficiency in solving the problem with the help of subproblems.

2. Optimal substructure : A problem shows optimal substructure if an optimal solution to the problem contains optimal solution to the sub-problems. In other words a problem has optimal substructure if the best next choice always leads to the optimal solution.

3.2 : Knapsack Problem

Q.4 Write an algorithm for Knapsack problem using Greedy strategy.

[SPPU : May-17, Marks 6]

Ans. :

Algorithm Knapsack_Greedy(W,n)

```
{  
//p[i] contains the profits of i items such that  $1 \leq i \leq n$   
//w[i] contains weights of I items.  
//x[i] is the solution vector.  
//W is the total size of Knapsack.
```

```
  for i : =1 to n do
```

```
  {  
if[w[i]<W] then//capacity of knapsack is a constraint
```

```
  {  
x[i] : =1.0;
```

```
W=W - w[i];
```

```
}
```

```
}
```

This is the basic operation of selecting x[i] lies within for loop.
 $\therefore$ Time complexity = $\Theta(n)$.


```

if(i <= n) then x[i] := W/w[i];
}

```

Q.5 Consider the following instances of the knapsack problem :
 $n = 3$, $m = 20$, $(P_1, P_2, P_3) = (24, 25, 15)$ and $(W_1, W_2, W_3) = (18, 15, 20)$ find the feasible solutions.

[SPPU : May-18, Marks 6, Dec.-13, Marks 8]

Ans. : We will first find the feasible solutions. From these feasible solutions we will find the solution that gives us maximum profit. For obtaining feasible solution we use 3 approaches and these are -

- 1) Select object with maximum profit.
- 2) Select object with minimum weight.
- 3) Select object with maximum $\frac{\text{profit}}{\text{weight}}$ ratio.

Method 1 : Select object with maximum profit.

Object	x1	x2	x3
P	24	25	15
W	18	15	20

$$m = 20$$

Selected Object	Profit	Weight	Remaining weight
x2	25	15	$20 - 15 = 5$
x1	$24 * \frac{5}{18} = \frac{20}{3}$	$18 * \frac{5}{18} = 5$	$5 - 5 = 0$

Hence Maximum profit = $25 + \frac{20}{3} = 31.66$ with solution $\left(x_2, \frac{x_1 * 5}{18}\right)$

Method 2 : Select object with minimum weight.

$$M = 20$$

Selected Object	Profit	Weight	Remaining weight
x2	25	15	$20 - 15 = 5$
x1	$24 * \frac{5}{18} = \frac{20}{3}$	$18 * \frac{5}{18} = 5$	$5 - 5 = 0$

Hence **Maximum profit** = $25 + \frac{20}{3} = 31.66$ with solution $\left(x_2, \frac{x_1 * 5}{18}\right)$

Method 3 : Select object with maximum $\frac{P}{W}$ ratio.

	x1	x2	x3
$\frac{P}{W}$	$\frac{4}{3}$	$\frac{5}{3}$	$\frac{3}{4}$

m = 20

Selected Object	Profit	Weight	Remaining weight
x2	25	15	20 - 15 = 5
x1	$24 * \frac{5}{18} = \frac{20}{3}$	$18 * \frac{5}{18} = 5$	5 - 5 = 0

Hence **Maximum profit** = $25 + \frac{20}{3} = 31.66$ with solution $\left(x_2, \frac{x_1 * 5}{18}\right)$

Thus by all the three methods we obtain the solution as $\left(x_2, \frac{5 \times x_1}{18}\right)$ with maximum profit as 31.66.

Q.6 Find an optimal solution for the following knapsack instance using greedy method : Number of objects n = 5, capacity of knapsack m = 100, profits = (10, 20, 30, 40, 50), weights = (20, 30, 66, 40, 60).

[SPPU : May-16, End Sem, Marks 8]

Ans. : We will first find the feasible solutions, From these feasible solutions we will find the solution that gives us maximum profit. For obtaining feasible solution we use 3 approaches and these are -

- 1) Select object with maximum profit.
- 2) Select object with minimum weight.
- 3) Select object with maximum $\frac{\text{profit}}{\text{weight}}$ ratio.

Method 1 : Select object with maximum profit.

Object	x1	x2	x3	x4	x5
p	10	20	30	40	50
w	20	30	66	40	60

$$m = 100$$

Selected object	Profit	Weight	Remaining weight
x5	50	60	$100 - 60 = 40$
x4	40	40	$40 - 40 = 0$

Hence Maximum profit = 90

with solution $(x_5, x_4) = (0,0,0,1,1)$

Method 2 : Select object with minimum weight.

$$m = 100$$

Selected object	Profit	Weight	Remaining weight
x1	10	20	$100 - 20 = 80$
x2	20	30	$80 - 30 = 50$
x4	40	40	$50 - 40 = 10$
x5	$50 \times \frac{1}{6} = 8.33$	$60 \times \frac{1}{6} = 10$	$10 - 10 = 0$

Hence Maximum profit = 78.33

with solution $(1,1,0,1,1/6)$

Method 3 : Select object with maximum profit/weight ratio.

$\frac{p}{w}$	x1	x2	x3	x4	x5
	$\frac{1}{2}$	$\frac{2}{3}$	$\frac{5}{11}$	1	$\frac{5}{6}$

$$m = 100$$

Selected object	Profit	Weight	Remaining weight
x4	40	40	$100 - 40 = 60$
x5	50	60	$60 - 60 = 0$

Maximum profit = 90

with solution (0,0,0,1,1)

This is a feasible solution.

Q.7 Consider the following instances of knapsack problem $n = 3$, $M = 50$, $(P_1, P_2, P_3) : (60, 100, 120)$ and $(W_1, W_2, W_3) : (10, 20, 30)$. Find the optimal solution using Greedy approach ?

👉 [SPPU : Dec.-19, Marks 6]

Ans. : We will use largest profit strategy to pick up the object. For that purpose, we will compute $\frac{P_i}{W_i}$.

x_i	1	2	3
P_i	60	100	120
W_i	10	20	30
$\frac{P_i}{W_i}$	6	5	4

Step 1 :

Select $x_i = 1$

Total Profit = 60

Remaining Weight = $M - W_1$
 $= 50 - 10$
 $= 40$

Step 2 :

Now Select $x_i = 2$

Total Profit = $60 + 100 = 160$

Remaining weight = $40 - 20$
 $= 20$

Step 3 :

We can not select items 3 as a whole. So, only select the weight which will fill up the knapsack completely.

∴ Take only wt 20 out of 30.

∴ Remaining weight = 20 - 20 = 0

$$\sum W_i x_i = \left(10 \times 1 + 20 \times 1 + 30 \times \frac{20}{30} \right) = 50$$

$$\sum p_i x_i = \left(60 \times 1 + 100 \times 1 + 120 \times \frac{20}{30} \right)$$

$$= 60 + 100 + 80$$

$$= 240$$

The optimal solution is $\left(1, 1, \frac{2}{3} \right)$

3.3 : Scheduling Algorithms - Job Scheduling

Q.8 Explain job-sequencing with deadlines. Solve the following instance : $n = 5$.

$(P_1, P_2, P_3, P_4, P_5) = (20, 15, 10, 5, 1)$

$(d_1, d_2, d_3, d_4, d_5) = (2, 2, 1, 3, 3)$ [SPPU : June-12, May-13, Marks 10]

Ans. :

Step 1 : We will arrange the profits P_i in descending order, along with corresponding deadlines.

Profit	20	15	10	5	1
Job	P_1	P_2	P_3	P_4	P_5
Deadline	2	2	1	3	3

Step 2 : Create an array $J []$ which stores the jobs. Initially $J []$ will be

1	2	3	4	5
0	0	0	0	0

$J [5]$

Step 3 : Add i^{th} Job in array $J[]$ at index denoted by its deadline D_i .
First job is P_1 , its deadline is 2.
Hence insert P_1 in the array $J[]$ at 2nd index.

1	2	3	4	5
	P_1			

Step 4 : Next job is P_2 whose deadline is 2 but $J[2]$ is already occupied. We will search for empty location $< J[2]$. The $J[1]$ is empty. Insert P_2 at $J[1]$.

Step 5 : Next job is P_3 whose deadline is 1. But as $J[1]$ is already occupied discard this job.

Step 6 : Next job is P_4 with deadline 3. Insert P_4 at $J[3]$.

1	2	3	4	5
P_2	P_1	P_4		

Step 7 : Next job is P_5 with deadline 3. But as there is empty slot at $\leq J[3]$, we will discard this job.

Step 8 : Thus the optimal sequence which we will obtain will be $P_2 - P_1 - P_4$. The maximum profit will be 40.

Q.9 Find the correct sequence for jobs using following instances.

Job	J1	J2	J3	J4	J5
Profit	20	15	10	5	1
Deadline	2	2	1	3	3

[SPPU : April-19, Marks 5]

Ans. :

Step 1 : We will arrange the profits P_i in descending order, along with corresponding deadlines.

Profit	20	15	10	5	1
Job	P_1	P_2	P_3	P_4	P_5
Deadline	2	2	1	3	3

Step 2 : Create an array $J[]$ which stores the jobs. Initially $J[]$ will be

1	2	3	4	5
0	0	0	0	0

$J[5]$

Step 3 : Add i^{th} Job in array $J[]$ at index denoted by its deadline D_i .
First job is P_1 , its deadline is 2.
Hence insert P_1 in the array $J[]$ at 2nd index.

1	2	3	4	5
	P_1			

Step 4 : Next job is P_2 whose deadline is 2 but $J[2]$ is already occupied. We will search for empty location $< J[2]$. The $J[1]$ is empty. Insert P_2 at $J[1]$.

Step 5 : Next job is P_3 whose deadline is 1. But as $J[1]$ is already occupied discard this job.

Step 6 : Next job is P_4 with deadline 3. Insert P_4 at $J[3]$.

1	2	3	4	5
P_2	P_1	P_4		

Step 7 : Next job is P_5 with deadline 3. But as there is empty slot at $\leq J[3]$, we will discard this job.

Step 8 : Thus the optimal sequence which we will obtain will be $P_2 - P_1 - P_4$. The maximum profit will be 40.

Q.10 Write an algorithm for Job scheduling.

Ans. :

Algorithm Job_seq(D, J, n)

{

//Problem description : This algorithm is for job sequencing using Greedy method.

// $D[i]$ denotes i^{th} deadline where $1 \leq i \leq n$

```

//J[i] denotes ith job
// D[J[i]] ≤ D[J[i+1]]
//Initially
D[0] ← 0;
J[0] ← 0;
J[1] ← 1;
count ← 1;
for ← 2 to n do
{
    t ← count;
    while(D[J[t]] > D[i]) AND D[J[t]] ≠ t) do t ← t-1;
    if((D[J[t]] ≤ D[i]) AND (D[i] > t)) then
    {
        //insertion of ith feasible sequence into J array
        for s ← count to (t+1) step -1 do
            J[s+1] ← J[s];
        J[t+1] ← i;
        count ← count+1;
    } //end of if
} //end of while
return count;
}

```

The sequence of J will be inserted if and only if $D[J[t]] \neq t$. This also means that the job J will be processed if it is in within the deadline.

The computing time taken by above Job_seq algorithm is $O(n^2)$, because the basic operation of computing sequence in array J is within two nested for loops.

3.4 : Activity Selection Problem

Q.11 Write greedy algorithm for activity selection problem. Give its time complexity. For following intervals, select the activities according to your algorithm. I_1 (1-3), I_2 (0-2), I_3 (3-6), I_4 (2-5), I_5 (5-8), I_6 (3-10), I_7 (7-9).

Ans. :

Step 1 : Sort the given activities in ascending order according to their finishing time.

Step 2 : Select the first activity from sorted array and add it to **solution** array.

Step 3 : For next subsequent activity if start time of currently selected activity is greater than or equal to finish time of previously selected activity, then add it to **solution** array.

Step 4 : Select next activity.

Step 5 : Repeat step 3 and step 4 for all remaining activities.
Time complexity is $O(n \log n)$.

Example :

Step 1 : We will sort the given activities in ascending order according to their finish time. The sorted table is

Start Time	Finish Time	Activity Name
0	2	I_2
1	3	I_1
2	5	I_4
3	6	I_3
5	8	I_5
7	9	I_7
3	10	I_6

Step 2 : Select first activity i.e. activity I_2 to add it to **solution** array.
Hence **Solution** = $\{I_2\}$.

Step 3 : Now select activity I_1 . If $\text{start}(I_1) < \text{finish}(I_2)$. So we will not add I_1 to **solution** array.

Step 4 : Now select activity I_4 . $\text{Start}(I_4) = \text{Finish}(I_4) = 2$. Add I_4 to solution array.

$\therefore$ Solution = $\{I_2, I_4\}$

Step 5 : Select I_3 . $\text{Start}(I_3) < \text{Finish}(I_5)$. Hence do not add it to solution array.

Step 6 : Similarly activities I_3, I_5, I_7 and I_6 will not get added to solution array.

Step 7 : Hence solution is $\{I_2, I_4\}$.

3.5 : Introduction to Dynamic Programming

Q.12 Name the elements of dynamic programming ? How does the dynamic programming solve the problem ?

☞ [SPPU : Dec.-06, 07, May-07, 08, Marks 6, May-10, Marks 4]

Ans. : Elements of Dynamic Programming :

- 1) **Optimal substructure** - The dynamic programming technique makes use of principle of optimality to find the optimal solution from subproblems.
- 2) **Overlapping subproblems** - The dynamic programming is a technique in which the problem is divided into subproblems. The solutions of subproblems are shared to get the final solution to the problem. It avoids repetition of work and we can get the solution more efficiently.

Steps of Dynamic Programming : Dynamic programming design involves 4 major steps :

1. Characterize the structure of optimal solution. That means develop a mathematical notation that can express any solution and subsolution for the given problem.
2. Recursively define the value of an optimal solution.
3. By using bottom up technique compute value of optimal solution. For that you have to develop a recurrence relation that relates a solution to its subsolutions, using the mathematical notation of step 1.
4. Compute an optimal solution from computed information.

Q.13 Write control abstraction (General Strategy Algorithm) of dynamic programming. [SPPU : Aug.-15, In Sem, Marks 4]

Ans. : Refer Q.12.

Q.14 What is the "Principle of optimality" ?

[SPPU : Dec.-06, 07, Marks 2]

Ans. : • The dynamic programming algorithm obtains the solution using principle of optimality.

- The principle of optimality states that "in an optimal sequence of decisions or choices, each subsequence must also be optimal."
- When it is not possible to apply the principle of optimality it is almost impossible to obtain the solution using the dynamic programming approach.
- **For example :** Finding of shortest path in a given graph uses the principle of optimality.

Q.15 Enlist the applications of Dynamic programming.

Ans. : Dynamic programming is used for solving various problems such as

1. Multistage Graph
2. Traveling Salesman Problem
3. Matrix Multiplication Problem
4. Longest Common Subsequence Problem
5. Maximum Flow Problem
6. All Pair Shortest Path Problem

Q.16 Comparison of greedy approach and dynamics programming.

[SPPU : Dec.-18, Marks 6]

Ans. :

Sr. No.	Greedy method	Dynamic programming
1.	Greedy method is used for obtaining optimum solution.	Dynamic programming is also for obtaining optimum solution.
2.	In Greedy method a set of feasible solutions and the picks up the optimum solution.	There is no special set of feasible solutions in this method.

3.	In Greedy method the optimum selection is without revising previously generated solutions.	Dynamic programming considers all possible sequences in order to obtain the optimum solution.
4.	In Greedy method there is no as such guarantee of getting optimum solution.	It is guaranteed that the dynamic programming will generate optimal solution using principle of optimality .

3.6 : Binomial Coefficients

Q.17 Explain with example, how to obtain Binomial coefficient using dynamic programming ?

Ans. : • Computing binomial coefficient is a typical example of applying dynamic programming.

• In mathematics, particularly in **combinatorics**, binomial coefficient is a coefficient of any of the terms in the expansion of $(a + b)^n$. It is

denoted by $C(n, k)$ or $\binom{n}{k}$ where $(0 \leq k \leq n)$.

• The **formula** for computing binomial coefficient is,

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$$

and $C(n, 0) = 1$

$$C(n, n) = 1$$

where $n > k > 0$

For example : Compute $C(4, 2)$

$$n = 4, k = 2$$

$$C(4, 2) = C(n - 1, k - 1) + C(n - 1, k)$$

$$C(4, 2) = C(3, 1) + C(3, 2) \quad \dots (Q.17.1)$$

As there are two unknowns : $C(3, 1)$ and $C(3, 2)$ in above equation we will compute these sub instances of $C(4, 2)$.

$$\therefore n = 3, k = 1$$

$$C(3, 1) = C(2, 0) + C(2, 1)$$

As $C(n, 0) = 1$ We can write

$$C(2, 0) = 1$$

$$\therefore C(3, 1) = 1 + C(2, 1)$$

... (Q.17.2)

Hence let us compute $C(2, 1)$.

$$n = 2, k = 1$$

$$\begin{aligned}\therefore C(2, 1) &= C(n - 1, k - 1) + C(n - 1, k) \\ &= C(1, 0) + C(1, 1)\end{aligned}$$

But as $C(n, 0) = 1$ and $C(n, n) = 1$ we get

$$C(1, 0) = 1 \text{ and } C(1, 1) = 1$$

$$\begin{aligned}\therefore C(2, 1) &= C(1, 0) + C(1, 1) \\ &= 1 + 1\end{aligned}$$

$C(2, 1) = 2$

... (Q.17.3)

Put this value in equation (3.6.2) and we get

$$C(3, 1) = 1 + 2$$

$C(3, 1) = 3$

... (Q.17.4)

Now to solve equation 1 we will first compute $C(3, 2)$ with $n = 3$ and $k = 2$.

$$\therefore C(3, 2) = C(n - 1, k - 1) + C(n - 1, k)$$

$$C(3, 2) = C(2, 1) + C(2, 2)$$

But as $C(n, n) = C(2, 2) = 1$, we will put values of $C(2, 1)$ [obtained in equation (3.6.3) and $C(2, 2)$ in $C(3, 2)$ we get,

$$\begin{aligned}C(3, 2) &= C(2, 1) + C(2, 2) \\ &= 2 + 1\end{aligned}$$

$C(3, 2) = 3$

... (Q.17.5)

Put equation (Q.17.4) and (Q.17.5) in equation (Q.17.1), then we get

$$\begin{aligned}C(4, 2) &= C(3, 1) + C(3, 2) \\ &= 3 + 3\end{aligned}$$

$$C(4, 2) = 6$$

is the final answer.

While computing $C(n, k)$ the smaller overlapping sequences get generated by $C(n - 1, k - 1)$ and $C(n - 1, k)$. These overlapping, smaller instances of problem need to be solved first. The solutions which we obtained by solving these instances will ultimately generate the final solution. Thus for computing binomial coefficient dynamic programming is used.

3.7 : OBST

Q.18 Write an algorithm for finding minimum cost binary search tree using dynamic programming strategy. Show that the computing time of this algorithm is $O(n^2)$. [SPPU : Dec.-06, May-07, 09, Marks 12]

OR Write an algorithm for optimum binary search tree.

[SPPU : Dec.-15, End Sem, Marks 6]

Ans. :

Algorithm Wt(p,q,W,n)

//p is an input array [1..n]

//q is an input array [0..n]

// W[i,j] will be output for this function such that W[0..n,0..n]

{

 for i ← 1 to n do

 W[i,i] ← q[i]; // initialize

 for m ← 1 to n do

 {

 for k ← 0 to n-m do

 {

 k ← i+m;

 W[i,k] ← W[i,k-1] + p[k] + q[k];

 }

 }

write(W[0:n]); //print the values of W

}

- For computing C_{ij} , r_{ij} following algorithm is used.

Algorithm OBST(p,q,W,C,r)

{

 //computation of first two rows

```

for i=0 to n do
{
  C[i,i] ← 0;
  r[i,i] ← 0;
  C[i,i+1] ← q[i]+q[i+1]+p[i+1];
  r[i,i+1] ← i+1;
}
for m ← 2 to n do
{
  For ← 0 to n-m do
  {
    j ← i+m;
    k ← Min_Value(C,r,i,j); //call for algorithm Min_Value
    // minimum value of Cij is obtained for deciding value of k
    C[i,j] ← W[i,j] + C[i,k-1] + C[k,j];
    r[i,j] ← k;
  }
}
write(C[0:n],r[0:n]); //print values of Cij and rij
}

```

Algorithm Min_Value(C,r,i,j)

```

{
  minimum ← ∞;
  // finding the range of k
  for (m ← r[i,j-1] to r[i+1,j]) do
  {
    if (C[i,m-1] + C[m,j]) < minimum then
    {
      minimum ← C[i,m-1] + C[m,j];
      k ← m;
    }
  }
  return k; //This k gives minimum value of C
}
}

```

Following algorithm is used for creating the tree T_{ij} .

Algorithm build_tree(r,a,i,j)

```

{
  T ← new(node); //allocate memory for a new node
  k ← r[i,j];
  T->data ← a[k];
  if (j==i+1)
    return;
  T->left=build_tree(r,a,i,k-1);
  T->right=build_tree(r,a,k,j);
}

```

In order to obtain the optimal binary search tree we will follow :

1. Compute the weight using W_t function.
2. Compute C_{ij} and r_{ij} using OBST.
3. Build the tree using build_tree function.

Analysis

In given algorithm, the basic operation is computation of $C[i, j]$. In this computation $j \leftarrow i + m$ i.e. $j - i = m$. The inner for loop executes for $n - m + 1$ times to compute $C[i, j]$. Inside this nested for loop a function Min-value is called. The purpose of this function is to compute minimum value of C array. The time complexity of this computation is $O(m)$.

Hence total time required by the algorithm to execute is $O(nm - m^2)$ which turns out to be $O(n^3)$. But D.E. Knuth has shown that if value of k is searched within the limit of $r[i, j, -1]$ to $r[i+1, j]$ then the time complexity of above algorithm will be $O(n^2)$.

Q.19 What is dynamic programming approach to solve the problem ?

$N = 3$ and $\{a_1, a_2, a_3\} = \{\text{do, if, while}\}$ Let $p(1:3) = (0.5, 0.1, 0.05)$
 $q(0:3) = (0.15, 0.1, 0.05, 0.05)$. Compute and construct OBST for above values using dynamic approach.

[SPPU : May-11, Marks 16, Aug.-15, In Sem, Marks 8]

Ans. : For convenience we multiply the probabilities p_L and q_i by 100.

Hence $p_i = (50, 10, 5)$, $q_i = (15, 10, 5, 5)$

$$W_{i,i} = q_i, \quad r_{i,i} = 0, \quad C_{i,i} = 0$$

$$W_{i,i+1} = q_i + q_{(i+1)} + P_{(i+1)}$$

$$r_{i,i+1} = i + 1$$

$$C_{i,i+1} = q_i + q_{(i+1)} + p_{(i+1)}$$

$$W_{i,j} = W_{i,j-1} + p_i + q_j$$

$$r_{i,j} = k$$

$$C_{i,j} = i < \min_k \leq j \left\{ C_{(i,k-1)} + C_{k,j} \right\} + W_{i,j}$$

We will construct the tables for values of W, C and r.

Let, $i = 0$

$$W_{00} = q_0 = 15$$

When $i = 1$,

$$W_{11} = q_1 = 10$$

When $i = 2$,

$$W_{22} = q_2 = 5$$

When $i = 3$,

$$W_{33} = q_3 = 5$$

When $i = 0$ and $j - i = 1$ then,

$$\begin{aligned} W_{01} &= q_0 + q_1 + p_1 \\ &= 15 + 10 + 50 \end{aligned}$$

$$W_{01} = 75$$

When $i = 1$ and $j - i = 2$ then,

$$\begin{aligned} W_{12} &= q_1 + q_2 + p_2 \\ &= 10 + 5 + 10 \end{aligned}$$

$$W_{12} = 25$$

When $i = 2$ and $j - i = 3$

$$\begin{aligned} W_{23} &= q_2 + q_3 + p_3 \\ &= 5 + 5 + 5 \end{aligned}$$

$$W_{23} = 15$$

When $i = 0, j - i = 2$

$$W_{ij} = W_{i,j-1} + p_j + q_j$$

$$\begin{aligned} W_{02} &= W_{01} + p_2 + q_2 \\ &= 75 + 10 + 5 \end{aligned}$$

$$W_{02} = 90$$

When $i = 1, j - i = 2$

$$\begin{aligned} W_{13} &= W_{12} + p_3 + q_3 \\ &= 25 + 5 + 5 \end{aligned}$$

$$W_{13} = 35$$

Now, when $i = 0, j - i = 3$ then

$$\begin{aligned} W_{03} &= W_{02} + p_3 + q_3 \\ &= 90 + 5 + 5 \end{aligned}$$

$$W_{03} = 100$$

The table for W can be represented as

	0	1	2	3
$j-i \downarrow$				
0	$W_{00} = 15$	$W_{11} = 10$	$W_{22} = 5$	$W_{33} = 5$
1	$W_{01} = 75$	$W_{12} = 25$	$W_{23} = 15$	
2	$W_{02} = 90$	$W_{13} = 35$		
3	$W_{03} = 100$			

We will compute for C and r

$$C_{i,i} = 0, r_{i,i} = 0$$

Hence $C_{00} = 0, C_{11} = 0, C_{22} = 0, C_{33} = 0$

$$r_{00} = 0, r_{11} = 0, r_{22} = 0, r_{33} = 0$$

$$C_{i,i+1} = q_i + q_{(i+1)} + p_{(i+1)} \text{ and } r_{i,i+1} = i+1$$

$\therefore$ When $i = 0$

$$\begin{aligned} C_{01} &= q_0 + q_1 + p_1 \\ &= 15 + 10 + 50 \\ &= 75 \end{aligned}$$

$$r_{01} = i + 1 = 1$$

When $i = 1$

$$\begin{aligned} C_{12} &= q_1 + q_2 + p_2 \\ &= 10 + 5 + 5 \end{aligned}$$

$$C_{12} = 20, r_{12} = 2$$

When $i = 2$

$$\begin{aligned} C_{23} &= q_2 + q_3 + p_3 \\ &= 5 + 5 + 5 \end{aligned}$$

$$C_{23} = 15, r_{23} = 3$$

Now we will compute C_{ij} and r_{ij} for $j - 1 \geq 2$.

$$\text{As } C_{ij} = i < \min_k \leq j \left\{ C_{(i,k-1)} + C_{k,j} \right\} + W_{i,j}$$

Hence we will find k .

For C_{02} we have $i = 0$ and $j = 2$. For $r_{i,j-1}$ to $r_{i+1,j}$ i.e. for r_{01} to r_{12} we will compute minimum value of $C_{i,j}$.

Let, $r_{01} = 1$ and $r_{12} = 2$. Then we will assume $k = 1$ and $k = 2$ and then compute $C_{i,j}$. From these computations we will pick-up the minimum value of $C_{i,j}$.

Let, with $k = 1, i = 0, j = 2$

$$C_{ij} = C_{i,k-1} + C_{k,j}$$

$$\begin{aligned} C_{02} &= C_{00} + C_{12} \\ &= 0 + 20 \end{aligned}$$

$$C_{02} = 20$$

with $k = 2, i = 0, j = 2$

$$\begin{aligned} C_{02} &= C_{i,k-1} + C_{k,j} \\ &= C_{01} + C_{22} \\ &= 75 + 0 \end{aligned}$$

$$C_{02} = 75$$

Clearly select $C_{02} = 20$ when $k = 1$.

$$\begin{aligned} \text{Now } C_{02} &= \min\{C_{02}\} + W_{02} \\ &= 20 + 90 \end{aligned}$$

$$C_{02} = 110$$

Now for C_{13} we have $i = 1$, and $j = 3$.

For $r_{i,j-1}$ to $r_{i+1,j}$ i.e. for r_{12} to r_{23} we will compute minimum value of $C_{i,j}$

Let $r_{12} = 2$ and $r_{23} = 3$. Then we will assume $k = 2$ and $k = 3$. then compute $C_{i,j}$

with $k = 2, i = 1, j = 3$

$$C_{ij} = C_{i, k-1} + C_{k, j}$$

$$C_{13} = C_{11} + C_{23}$$

$$= 0 + 15$$

$$C_{13} = 15$$

Clearly select $C_{13} = 15$ when $k = 2$

$$\text{Now } C_{13} = \min \{C_{13}\} + W_{13}$$

$$= 15 + 35$$

$$C_{13} = 50$$

Now, For C_{03} we have $i = 0$, and $j = 3$

For $r_{i, j-1}$ to $r_{i+1, j}$ i.e. for r_{02} to r_{13} i.e. with $k = 1$ to $k = 2$.

with $k = 1, i = 0, j = 3$

$$C_{ij} = C_{i, k-1} + C_{k, j}$$

$$= C_{00} + C_{13}$$

$$= 0 + 15$$

$$C_{03} = 15$$

Clearly select $C_{03} = 15$ when $k = 1$

$$\text{Now } C_{03} = \min \{C_{03}\} + W_{03}$$

$$= 15 + 100$$

$$C_{03} = 115$$

The C_{ij} and r_{ij} table can be as given below,

	0	1	2	3
0	$C_{00} = 0$ $r_{00} = 0$	$C_{11} = 0$ $r_{11} = 0$	$C_{22} = 0$ $r_{22} = 0$	$C_{33} = 0$ $r_{33} = 0$
1	$C_{01} = 75$ $r_{01} = 1$	$C_{12} = 20$ $r_{12} = 2$	$C_{23} = 15$ $r_{23} = 3$	
2	$C_{02} = 110$ $r_{02} = 1$	$C_{13} = 50$ $r_{13} = 2$		
3	$C_{03} = 115$ $r_{03} = 1$			

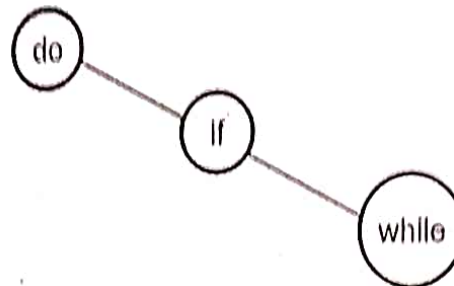
The tree T_{03} has a root r_{03} . The $r_{03} = 1$. That means 01 is the root.

The $r_{1k} = 1$ becomes left child and r_k becomes the right child. The value of $k = 1$.

∴ $r_{00} = 0$ i.e. empty left child

$r_{13} = 2$ i.e. a_2 is right

Finally the tree will be -



3.8 : The 0/1 Knapsack

Q.20 Explain 0/1 Knapsack using dynamic programming with an example.

[SPPU : May-14, Marks 8]

Ans. :

Problem Description : If we are given n objects and a Knapsack or a bag in which the object i that has weight w_i is to be placed. The Knapsack has a capacity W . Then the profit that can be earned is $p_i x_i$. The objective is to obtain filling of Knapsack with maximum profit earned.

$$\text{maximized } \sum_n p_i x_i \text{ subject to constraint } \sum_n w_i x_i \leq W$$

where $1 \leq i \leq n$ and n is total number of objects. And $x_i = 0$ or 1

To solve Knapsack problem using dynamic programming we will write the recurrence relation as :

$$\text{table}[i, j] = \text{maximum} \{ \text{table}[i-1, j], v_i + \text{table}[i-1, j-w_i] \} \text{ if } j \geq w_i$$

or

$$\text{table}[i-1, j] \text{ if } j < w_i$$

That means, a table is constructed using above given formula. Initially, table $[0, j] = 0$ as well as table $[i, 0] = 0$ when $j \geq 0$ and $i \geq 0$.

The initial stage of the table can be

	0	1	...	$j = w_i$	...	j	...	W
0	0	0	...	0	...	0	...	0
$\vdots$				table $[i-1, j-w_i]$				
$i-1$	0					table $[i-1, j]$		
i	0					table $[i, j]$		
$\vdots$								
n	0							table $[n, W]$

Goal i.e.,
maximum
value of
items

The table $[n, W]$ is a goal i.e., it gives the total items sum of all the selected items for the knapsack.

From this goal value the selected items can be traced out.

Example : Refer Q.21.

Q.21 Consider 0/1 knapsack problem $N = 3$, $w = (4, 6, 8)$, $p = (10, 12, 15)$ using dynamic programming devise the recurrence relations for the problem and solve the same. Determine the optimal profit for the knapsack of capacity 10. [SPPU : Dec.-11, 12, Marks 10]

Ans. : The recurrence relation would be

$$\text{table}[i, j] = \begin{cases} \max\{\text{table}[i-1, j], p_i + \text{table}[i-1, j-w_i]\} & \text{when } j \geq w_i \\ \text{or} \\ \text{table}[i-1, j], & \text{if } j < w_i \end{cases}$$

initially table $[0, j] = 0$ and table $[i, 0] = 0$.

There will be 0 to n rows and 0 to W columns in table.

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0										
2	0										
3	0										

Now we will fill up table row by row

table [1,1] with $i = 1$, $j = 1$, $w_i = 4$, $p_i = 10$

As $j < w_i$

$$\begin{aligned}\text{table}[1,1] &= \text{table}[i-1, j] \\ &= \text{table}[0,1]\end{aligned}$$

$\therefore \text{table}[1,1] = 0$

table [1,2] with $i = 1$, $j = 2$, $w_i = 4$, $p_i = 10$

As $j < w_i$

$$\begin{aligned}\text{table}[1,2] &= \text{table}[i-1, j] \\ &= \text{table}[0,2]\end{aligned}$$

$\text{table}[1,2] = 0$

Thus $\text{table}[1,3] = 0$

Now table [1,4] with $i = 1$, $j = 4$, $w_i = 4$, $p_i = 10$ is

As $j = w_i$

$$\text{table}[i, j] = \max\{\text{table}[i-1, j], p_i + \text{table}[i-1, j-w_i]\}$$

$$\text{table}[1,4] = \max\{\text{table}[0,4], 10 + \text{table}[0,0]\}$$

$\text{table}[1,4] = 10$

table [1,5] with $i = 1$, $j = 5$, $w_i = 4$, $p_i = 10$.

$$\text{table}[i, j] = \max\{\text{table}[i-1, j], p_i + \text{table}[i-1, j-w_i]\}$$

$$= \max(\text{table}[0,5], 10 + \text{table}[0,1])$$

$\text{table}[1,5] = 10$

Similarly $\text{table}[1,6] = \text{table}[1,7] = \text{table}[1,8] = \text{table}[1,9] = \text{table}[1,10] = 10$

The table will then be

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	10	10	10	10	10	10	10
2											
3											

table $[2,1]$ with $i = 2, j = 1, w_2 = 6, p_2 = 12$

As $j < w_2$

$$\text{table}[2,1] = \text{table}[1,1]$$

$$\text{table}[2,1] = 0$$

Similarly $\text{table}[2,2] = \text{table}[2,3] = \text{table}[2,4] = \text{table}[2,5] = 0$

Now table $[2,6]$ with $i = 2, j = 6, w_2 = 6, p_2 = 12$.

As $j = w_2 = 6$

$$\begin{aligned} \text{table}[2,6] &= \max\{\text{table}[i-1,j], p_i + \text{table}[i-1,j-w_i]\} \\ &= \max\{\text{table}[1,6], 12 + \text{table}[1,0]\} \end{aligned}$$

$$\text{table}[2,6] = 12$$

Similarly $\text{table}[2,7] = \text{table}[2,8] = \text{table}[2,9] = 12$

Now with table $[2,10]$; $i = 2, j = 10, w_2 = 6, p_2 = 12$.

$$\begin{aligned} \text{table}[2,10] &= \max\{\text{table}[i-1,j], p_i + \text{table}[i-1,j-w_i]\} \\ &= \max\{\text{table}[1,10], 12 + \text{table}[1,4]\} \end{aligned}$$

$$\text{table}[2,10] = 22$$

The table will then be

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	10	10	10	10	10	10	10
2	0	0	0	0	0	0	12	12	12	12	22
3	0										

table $[3, 1]$ with $i = 3, j = 1, w_3 = 8, p_3 = 15$.

$j < w_i$ is true for $j = 1$ to 7 .

Hence

$$\text{table}[3, 1] = \text{table}[2, 1] = 0$$

$$\text{table}[3, 2] = \text{table}[2, 2] = 0$$

$$\text{table}[3, 3] = \text{table}[2, 3] = 0$$

$$\text{table}[3, 4] = \text{table}[3, 5] = 0$$

$$\text{table}[3, 6] = \text{table}[2, 6] = 12$$

$$\text{table}[3, 7] = \text{table}[2, 7] = 12$$

Now with table $[3, 8]$ $i = 3, j = 8, w_3 = 8, p_3 = 15$.

As $j = w_i$

$$\begin{aligned} \text{table}[3, 8] &= \max\{\text{table}[i-1, j], p_i + \text{table}[i-1, j-w_i]\} \\ &= \max\{\text{table}[2, 8], 15 + \text{table}[2, 0]\} \end{aligned}$$

$$\text{table}[3, 8] = 15$$

$$\text{Similarly table}[3, 9] = 15$$

But with table $[3, 10]$ $i = 3, j = 10, w_3 = 8, p_3 = 15$.

$$\text{table}[3, 10] = \max\{\text{table}[2, 10], 15 + \text{table}[2, 2]\}$$

$$\text{table}[3, 10] = 22$$

The table will be

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	10	10	10	10	10	10	10	10
2	0	0	0	0	0	0	12	12	12	12	22
3	0	0	0	0	0	0	12	12	15	15	22

Now let us apply steps to identify selected items.

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	10	10	10	10	10	10	10	10
2	0	0	0	0	0	0	12	12	12	12	22
3	0	0	0	0	0	0	12	12	15	15	22

$i = 3, k = 10$

check table $[i, k] \stackrel{?}{=} \text{table}[i-1, k]$

check if table $[3, 10] = \text{table}[2, 10]$

i.e. $22 = 22$

$\therefore$ Do not select i^{th} i.e. 3^{rd} item.

Now set $i = i - 1$

$\therefore i = 3 - 1 = 2$

Now $i = 2, k = 10$

$\therefore \text{table}[2, 10] \stackrel{?}{=} \text{table}[1, 10]$

As table $[2, 10] \neq \text{table}[1, 10]$

i.e. $10 \neq 0$

$\therefore$ Select i^{th} i.e. 2^{nd} item.

Set $i = i - 1$ and $k = k - w_i$

$\therefore i = 2 - 1 = 1$ and $k = 10 - 6 = 4$

Now $i = 1, k = 4$

$\therefore \text{table}[1, 4] \stackrel{?}{=} \text{table}[0, 4]$

As $10 \neq 0$

Select i^{th} i.e. 1^{st} item

Set $i = i - 1$ and $k = k - w_i$

$\therefore i = 1 - 1 = 0$ and $k = 4 - 4 = 0$

Terminate the process by selecting item 1 and item 2

3.9 : Chain Matrix Multiplication

Ans. : We will compute matrix chain order.

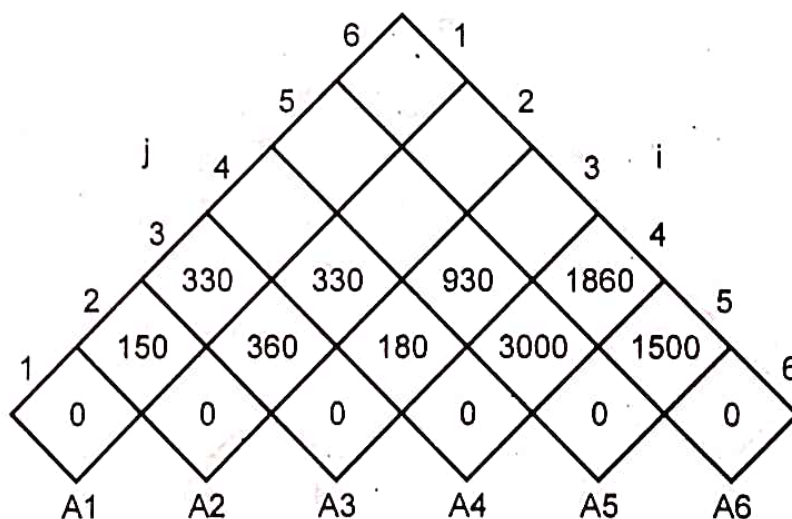
Hence $m[1, 1] = m[2, 2] = m[3, 3] = m[4, 4] = 0$.

$$m[5, 6] = P_4 * P_5 * P_6 = 5 * 50 * 6 = 1500$$

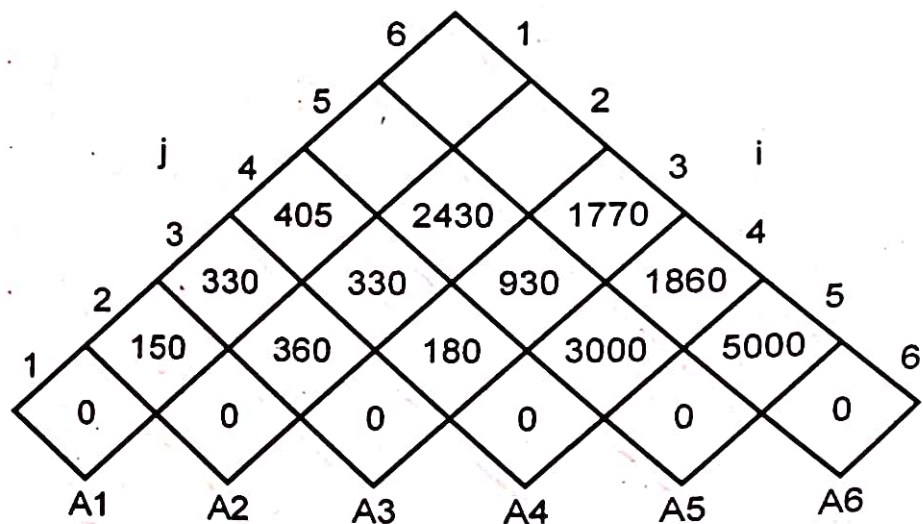
Now we will fill up the table for the values $m[1, 3]$, $m[2, 4]$, $m[3, 5]$ and $m[4, 6]$.

$m[1, 3] = \min$	$m[1,1] + m[2,3] + P_0 P_1 P_3$	$= 0 + 360 + (5 \cdot 10 \cdot 12)$ $= 960$	min = 330 with $k = 2$
	$m[1,2] + m[3,3] + P_0 P_2 P_3$	$= 150 + 0 + (5 \cdot 3 \cdot 12)$ $= 330$	
$m[2, 4] = \min$	$m[2,2] + m[3,4] + P_1 P_2 P_4$	$= 0 + 180 + (10 \cdot 3 \cdot 5)$ $= 330$	min = 330 $k = 2$
	$m[2,3] + m[4,4] + P_1 P_3 P_4$	$= 360 + 0 + (10 \cdot 12 \cdot 50)$ $= 6360$	
$m[3,5] = \min$	$m[3,3] + m[4,5] + P_2 P_3 P_5$	$= 0 + 3000 + (3 \cdot 12 \cdot 50)$ $= 4800$	min = 930 $k = 4$
	$m[3,4] + m[5,5] + P_2 P_4 P_5$	$= 180 + 0 + (3 \cdot 5 \cdot 50)$ $= 930$	
$m[4, 6] = \min$	$m[4,4] + m[5,6] + P_3 P_4 P_6$	$= 0 + 1500 + (12 \cdot 5 \cdot 6)$ $= 1860$	min = 1860 $k = 4$
	$m[4,5] + m[6,6] + P_3 P_5 P_6$	$= 3000 + 0 + (12 \cdot 50 \cdot 6)$ $= 6600$	

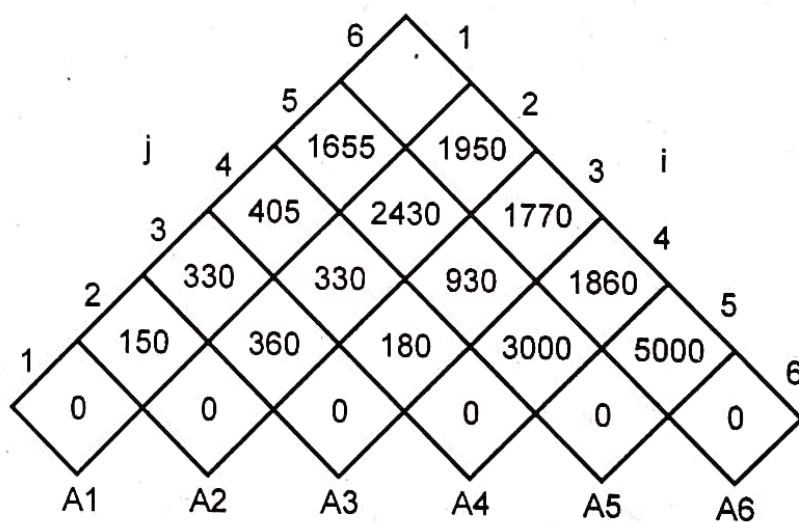
Now we will compute $m[1, 4]$, $m[2, 5]$ and $m[3, 6]$



$m[1, 4] = \min$	$m[1,1] + m[2,4] + P_0 P_1 P_4$	$= 0 + 330 + (5 \cdot 10 \cdot 5)$ $= 580$	$\min = 405$ $k = 2$
	$m[1,2] + m[3,4] + P_0 P_2 P_4$	$= 150 + 180 + (5 \cdot 3 \cdot 5)$ $= 405$	
	$m[1,3] + m[4,4] + P_0 P_3 P_4$	$= 330 + 0 + (5 \cdot 12 \cdot 5)$ $= 630$	
$m[2, 5] = \min$	$m[2,2] + m[3,5] + P_1 P_2 P_5$	$= 0 + 930 + (10 \cdot 3 \cdot 50) =$ 2430	$\min = 2430$ $k = 2$
	$m[2,3] + m[4,5] + P_1 P_3 P_5$	$= 360 + 3000 +$ $(10 \cdot 12 \cdot 50) = 9360$	
	$m[2,4] + m[5,5] + P_1 P_4 P_5$	$= 330 + 0 + (10 \cdot 5 \cdot 50)$ $= 2830$	
$m[3, 6] = \min$	$m[3,3] + m[4,6] + P_2 P_3 P_6$	$= 0 + 1860 + (3 \cdot 12 \cdot 6)$ $= 2076$	$\min = 1770$ $k = 4$
	$m[3,4] + m[5,6] + P_2 P_4 P_6$	$= 180 + 1500 + (3 \cdot 5 \cdot 6)$ $= 1770$	
	$m[3,5] + m[6,6] + P_2 P_5 P_6$	$= 930 + 0 + (3 \cdot 50 \cdot 6)$ $= 1830$	



$m[1, 5] = \min$	$m[1, 1] + m[2, 5] + P_0 P_1 P_5$	$= 0 + 2430 + (5 \cdot 10 \cdot 50)$ $= 4930$	$\min = 1655$ $k = 4$
	$m[1, 2] + m[3, 5] + P_0 P_2 P_5$	$= 150 + 930 + (5 \cdot 3 \cdot 50)$ $= 1830$	
	$m[1, 3] + m[4, 5] + P_0 P_3 P_5$	$= 330 + 3000 + (5 \cdot 12 \cdot 50)$ $= 6330$	
	$m[1, 4] + m[5, 5] + P_0 P_4 P_5$	$= 405 + 0 + (5 \cdot 5 \cdot 50)$ $= 1655$	
$m[2, 6] = \min$	$m[2, 2] + m[3, 6] + P_1 P_2 P_6$	$= 0 + 1770 + (10 \cdot 3 \cdot 6)$ $= 1950$	$\min = 1950$ $k = 2$
	$m[2, 3] + m[4, 6] + P_1 P_3 P_6$	$= 360 + 1860 + (10 \cdot 12 \cdot 6)$ $= 2940$	
	$m[2, 4] + m[5, 6] + P_1 P_4 P_6$	$= 330 + 1500 + (10 \cdot 5 \cdot 6)$ $= 2130$	
	$m[2, 5] + m[6, 6] + P_1 P_5 P_6$	$= 2430 + 0 + (10 \cdot 50 \cdot 6)$ $= 5430$	



The optimal parenthesization will be

$$(A1 * A2) * ((A3 * A4) * (A5 * 6))$$

The $s[1, 6] = 2$

$$\therefore (A1 \ A2) \ (A3 \ A4 \ A5 \ A6)$$

The $s[3, 6] = 4$

$$\therefore (A1 \ A2) \ (A3 \ A4) \ (A5 \ A6)$$

Thus we obtain optimal parenthesization as -

$$(A1 * A2) * (A3 * A4) * (A5 * A6)$$

END... ✍

Unit IV

4

Backtracking and Branch-n-Bound

4.1 : Principle

Q.1 Enlist the characteristics of backtracking strategy.

[SPPU : May-08, Marks 2]

Ans. : • In the backtracking method

1. The desired solution is expressible as an n tuple $(x_1, x_2, \dots, x_n)$ where x_i is chosen from some finite set S_i .
2. The solution maximizes or minimizes or satisfies a criterion function C
 $(x_1, x_2, \dots, x_n)$.

- The problem can be categorized into three categories.

For instance - For a problem P let C be the set of constraints for P . Let D be the set containing all solutions satisfying C then

Finding whether there is any feasible solution ? - Is the decision problem.

What is the best solution ? - Is the optimization problem.

Listing of all the feasible solution - Is the enumeration problem.

- The basic idea of backtracking is to build up a vector, one component at a time and to test whether the vector being formed has any chance of success.
- The major **advantage** of backtracking algorithm is that we can realize the fact that the partial vector generated does not lead to an optimal solution. In such a situation that vector can be ignored.
- Backtracking algorithm determines the solution by systematically searching the solution space (i.e. set of all feasible solutions) for the given problem.
- Backtracking is a depth first search with some bounding function.

Q.2 What are the constraints that must be satisfied while solving any problem using backtracking ? Explain briefly.

☞ [SPPU : Dec.-06, Marks 4; Dec.-10, Marks 6]

Ans. : • Backtracking algorithm solves the problem using two types of constraints -

Explicit constraint and Implicit constraints.

- **Definition :** Explicit constraints are the rules that restrict each element x_i has to be chosen from given set only. Explicit constraints depends on particular instance I of the problem. All the tuples from solution set must satisfy the explicit constraints.
- **Definition :** Implicit constraints are the rules that decide which tuples in the solution space of I satisfy the criterion function. Thus the implicit constraints represent by which x_i in the solution set must be related with each other.

Q.3 Explain implicit and explicit constraints.

☞ [SPPU : May-14, Marks 6]

OR What are implicit and explicit constraints with respect to backtracking ?

☞ [SPPU : May-14, Marks 8]

Ans. : Refer Q.2.

Q.4 Define following terms : State space, explicit constraints, implicit constraints, problem state, solution states, answer states, live node, E-node, dead node, bounding functions.

☞ [SPPU : May-11, Marks 8, Dec.-11, Marks 16, Dec.-12, Marks 18]

Ans. :

State space : All paths from root to other nodes define the state space of the problem. The tree organization for 4-queen's problems is shown by given Fig. Q.4.1. (Refer Fig. Q.4.1 on next page)

Explicit constraints : Explicit constraints are rules, which restrict, each vector element to be chosen from given set.

Implicit constraints : Implicit constraints are rules which determine which of the tuples in the solution space satisfy the criterion function.

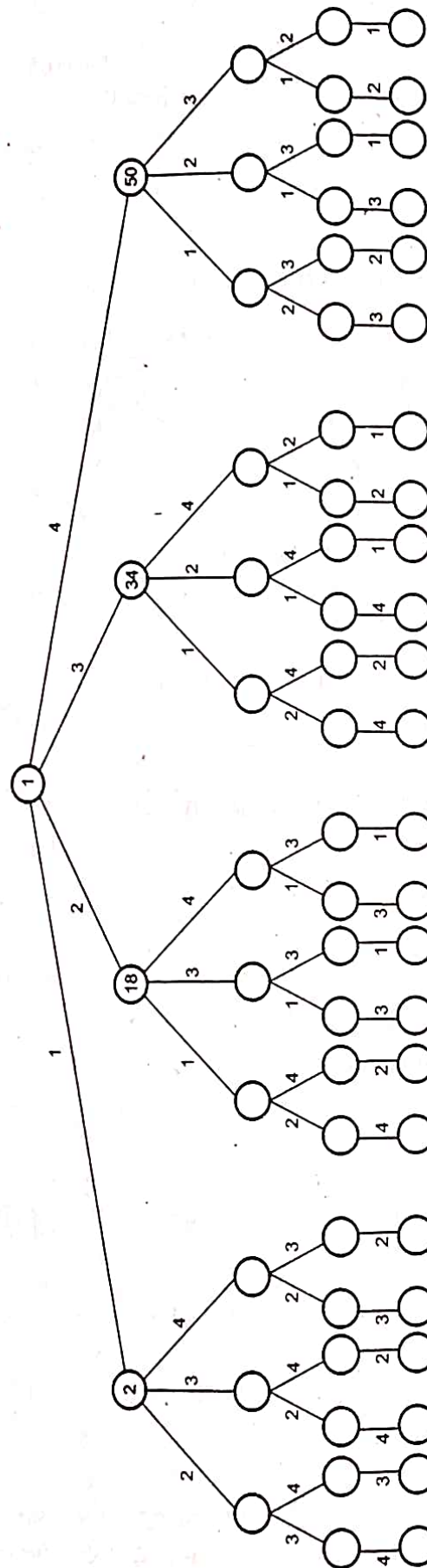


Fig. Q.4.1 State space tree for 4-queen's problems

Problem states : Each node in the state space tree is called problem state.

Solution states : The solution states are those problem states s for which the path from root to s defines a tuple in the solution space. In some trees the leaves define solution states.

Answer states : These are the leaf nodes which correspond to an element in the set of solutions. These are the states which satisfy the implicit constraints.

Live node : A node which is generated and whose children have not yet been generated is called live node.

E-node : The live node whose children are currently being expanded is called E-node. These are also called expanding nodes.

Dead node : A dead node is a generated node which is not to be expanded further or all of whose children have been generated.

Bounding functions : Bounding functions will be used to kill live nodes without generating all their children. A care should be taken while doing so, because atleast one answer node should be produced or even all the answer nodes be generated if problem needs to find all possible solutions.

Q.5 Explain the following terms : Live nodes, expanding nodes, bounding function and solution space. [SPPU : Dec.-13, Marks 8]

Ans. : Refer Q.4.

4.2 : Control Abstraction and Time Analysis

Q.6 Write a recursive algorithm which shows a recursive formulation of the backtracking technique. [SPPU : Dec.-06, Marks 8]

Ans. :

Algorithm Backtrack()

//This is recursive backtracking algorithm

//a[k] is a solution vector. On entering (k-1) remaining next

```

//values can be computed.
//T(a1,a2,...ak) be the set of all values for a(i+1), such that
//(a1,a2,...ai+1) is a path to problem state.
// Bi+1 is a bounding function such that if Bi+1(a1,a2,...ai+1) is false
//for a path (a1,a2,...ai+1) from root node to a problem state then
//path can not be extended to reach an answer node
{
  for ( each ak that belongs to T((a1,a2,...ak-1) do
  {
    if(Bk(a1,a2,...ak) = true) then // feasible sequence
    {
      if ((a1,a2,...ak) is a path to answer node then
        print(a[1],a[2],...a[k]);
      if (k<n) then
        Backtrack(k+1); //find the next set.
    }
  }
}

```

Q.7 Write a schema for an iterative backtracking method.

[SPPU : May-07, Dec.-07, Marks 8]

Ans. : The non recursive or iterative backtracking algorithm can be given as -

Algorithm Non_Rec_Back(n)

// This is a non recursive version of backtracking

//a[1,...,n] is a solution vector and each a₁,a₂,...a_k will be used to print solution.

```

{
  k:=1;
  while(k ≠ 0) do
  {
    if(any a[k] that belongs to T(a[1],a[2],...a[k-1])remains untried)
    AND (Bk (a[1],a[2],...,a[k] is true) then
    {
      if((a[1],a[2]...a[k]) is a path to answer node) then

```

```
    write(a[1],a[2],...a[k]); // solution printed
    //consider next element of the set
    k:=k+1;
else
    k :=k-1; //backtrack to most recent value
}
}
}
```

4.3 : Applications of Backtracking

Q.8 Enlist the applications of backtracking

Ans. : Various applications that are based on backtracking method are -

1. 8 Queen's problem : This is a problem based on chess games. By this problem, it is stated that arrange the queens on chessboard in such a way that no two queens can attack each other.
2. Sub of subset problem.
3. Graph coloring problem.
4. Finding Hamiltonian cycle.
5. Knapsack problem.

4.4 : The 8-Queen Problem

Q.9 Explain how backtracking strategy can be used to solve n-queen's problem. Give the pseudocode for the same. Discuss the time complexity for this problem. [SPPU : Dec.-07, Marks 12]

Ans. : Problem Statement : The n-queen's problem can be stated as follows.

Consider a $n \times n$ chessboard on which we have to place n queens so that no two queens attack each other by being in the same row or in the same column or on the same diagonal.

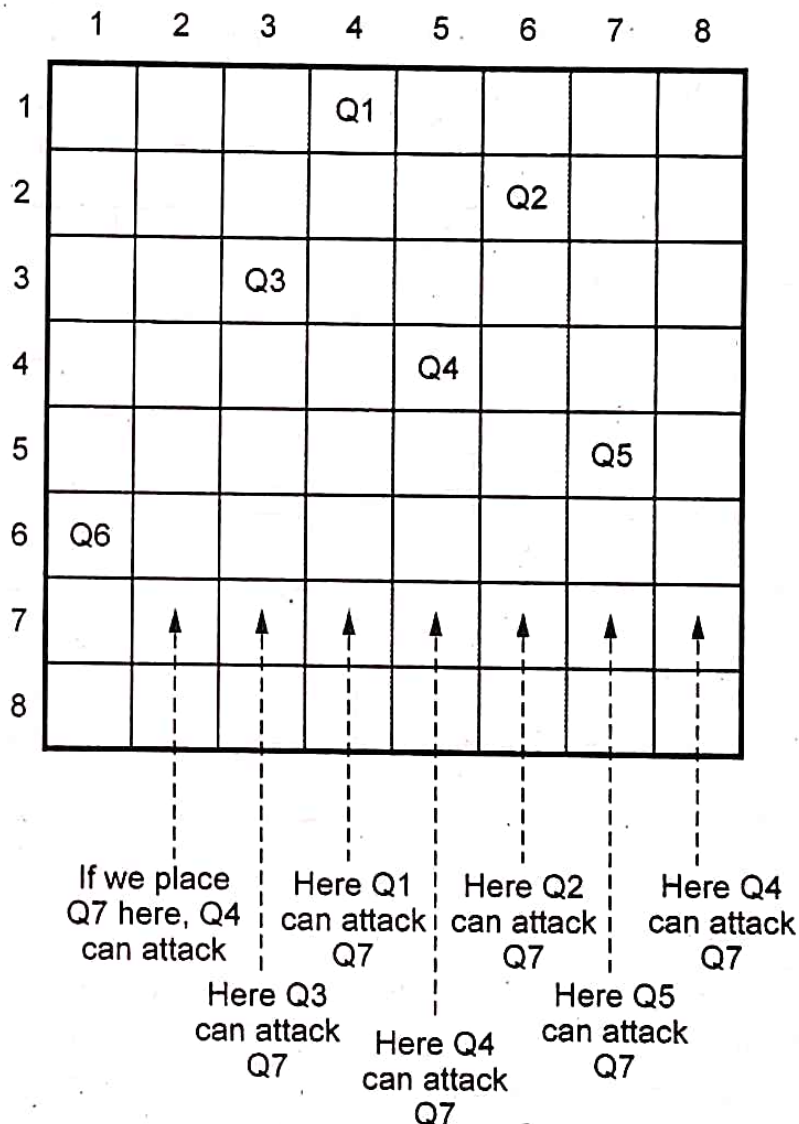
Example : Initially the chessboard is empty.

	1	2	3	4	5	6	7	8
1								
2								
3								
4								
5								
6								
7								
8								

Now we will start placing the queens on the chessboard.

	1	2	3	4	5	6	7	8
1		Q1						
2				Q2				
3	Q3							
4			Q4					
5					Q5			
6								
7								
8								

Thus we have placed 5 queens in such a way that no two queens can attack each other. Now if we want to place Q6 at location (6, 6) then queen Q5 can attack, if Q6 is placed at (6, 7) then Q1 can attack, if Q6 is placed at (6, 8) then Q2 can attack it. Similarly at (6, 5) the Q5 will attack it. At (6, 4) the Q2 will attack, at (6, 3) the Q4 will attack, at (6, 2) the Q1 will attack and at (6, 1) Q3 will attack the queen Q6. This shows that we need to backtrack and change the previously placed queens positions. It could then be -



Hence we have to backtrack to adjust already placed queens.

	1	2	3	4	5	6	7	8
1	Q1							
2								Q2
3						Q3		

4			Q4					
5							Q5	
6		Q6						
7				Q7				
8								

But again Q8 cannot be placed at any empty location safely. Hence we need to backtrack. Finally the successful placement of all the eight queens can be shown by following figure.

	1	2	3	4	5	6	7	8
1			Q1					
2						Q2		
3		Q3						
4							Q4	
5	Q5							
6				Q6				
7								Q7
8					Q8			

Algorithm

Algorithm Queen(n)

//Problem description : This algorithm is for implementing n queen's problem

//Input : total number of queen's n.

for column $\leftarrow 1$ to n do

{
if(place(row,column))then

{
board[row]column//no conflict so place queen

if(row=n)then//dead end

This function checks if two queens are on the same diagonal or not.

```

print_board(n)
    //printing the board configuration
else//try next queen with next position.
    Queen(row+1,n)
}

```

Row by row each queen is placed by satisfying constraints.

Algorithm place(row,column)

//Problem Description : This algorithm is for placing the
 //queen at appropriate position
 //Input : row and column of the chessboard
 //output : returns 0 for the conflicting row and column
 //position and 1 for no conflict.

```

for i ← 1 to row-1 do
{ //checking for column and diagonal conflicts
  if(board[i] = column)then
    return 0
  else if(abs(board[i]- column) = abs(i - row))then
    return 0
}
//no conflicts hence Queen can be placed
return 1

```

Same column by 2 queen's

This formula gives that 2 queens are on same diagonal

C Functions

/* By this function we try the next free slot
 and check for proper positioning of queen
 */

```

void Queen(int row,int n)
{
  int column;
  for(column=1;column<=n;column++)
  {
    if(place(row,column))
    {
      board[row] = column;//no conflict so place queen
      if(row==n)//dead end
        print_board(n);
    }
  }
}

```

```

        //printing the board configuration
    else //try next queen with next position
        Queen(row+1,n);
    }
}

int place(int row,int column)
{
    int i;
    for(i=1;i<=row-1;i++)
    { //checking for column and diagonal conflicts
        if(board[i] == column)
            return 0;
        else
            if(abs(board[i] - column) == abs(i - row))
                return 0;
    }
    //no conflicts hence Queen can be placed
    return 1;
}

```

The time complexity is solving n-queen's problem is $O(n!)$

Q.10 Write an algorithms to solve 8-queens problem using backtracking. [SPPU : May-11, Dec.-14, Marks 8]

Ans. : Refer Q.9.

Q.11 Explain how to solve 4 queen's problem using backtracking.

Ans. : • We start with empty chessboard.

- Place queen 1 in the first possible position of its row i.e. on 1st row and 1st column.

Q			

- Then place queen 2 after trying unsuccessful place - 1(1, 2), (2, 1), (2, 2) at (2, 3) i.e. 2nd row and 3rd column.

Q			
		Q	

- This is the dead end because a 3rd queen cannot be placed in next column, as there is no acceptable position for queen 3. Hence algorithm **backtracks** and places the 2nd queen at (2, 4) position.

Q			
			Q

- The place 3rd queen at (3, 2) but it is again another dead end as next queen (4th queen) cannot be placed at permissible position.

Q			
			Q
	Q		

- Hence we need to backtrack all the way upto queen 1 and move it to (1, 2).
- Place queen 1 at (1, 2), queen 2 at (2, 4), queen 3 at (3, 1) and queen 4 at (4, 3).

	Q		

	Q		
			Q

	Q		
			Q
Q			

	Q		
			Q
Q			
			Q

Thus solution is obtained.
(2, 4, 1, 3) in rowwise manner.

The state space tree of 4-queen's problem is shown in Fig. Q.11.1 (See on next page)

Q.12 Write an algorithm to solve 4 queen's problem using backtracking method. Use mathematical modelling to support your answer.

[SPPU : Dec.-13, Marks 8, Aug.-15, In Sem, Marks 6]

Ans. : 4 queen's problem : Refer Q.11.

Algorithm : Refer Q.9.

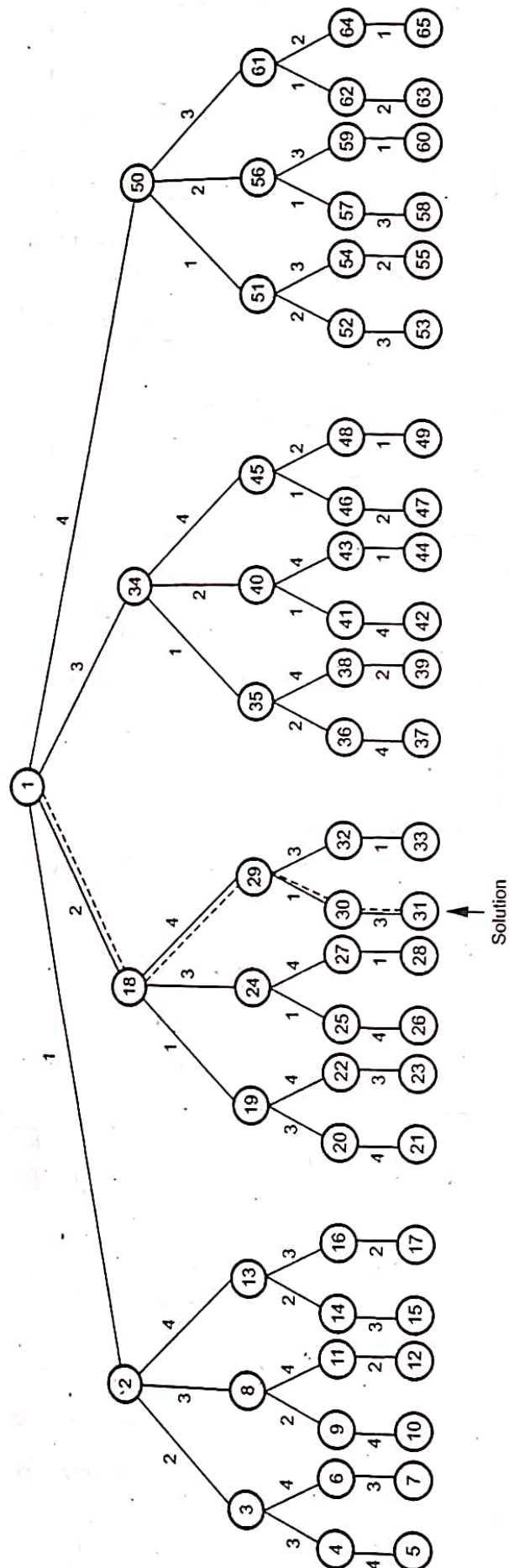


Fig. Q.11.1 State space tree for 4-queen's problem

Q.13 For a feasible sequence (7, 5, 3, 1) solve 8 queen's problem using backtracking.

[SPPU : May-14, Marks 10]

Ans. : While placing the queen at next position we have to check whether the current position chosen is on the same diagonal of previous queen's position.

If $P_1 = (i, j)$ and $P_2 = (k, l)$ are two positions then P_1 and P_2 lie on the same diagonal if $i + j = k + l$ or $i - j = k - l$. Let us put the given feasible sequence and try out remaining positions.

(1), 2, (3), 4, (5), 6, (7), 8 → Already occupied position.

j →		1	2	3	4	5	6	7	8	
i. values i.e. row values	7	5	3	1						
	7	5	3	1	2					The remaining vacant position is 2. But $4 - 1 = 5 - 2$. Not feasible.
	7	5	3	1	4					No conflict.
	7	5	3	1	4	6				$(3, 3) = (6, 6)$ ∴ Conflict occurs.
	7	5	3	1	4	8				List ends and we can not place further queen's.
	7	5	3	1	4					Hence backtrack.
	7	5	3	1	6					The next free slot 6 is tried.
	7	5	3	1	6	2				Not feasible because $(7, 1) = (2, 6)$ $1 + 7 = 6 + 2$. That is on same diagonal.
	7	5	3	1	6	4				Occupied are circled.
	7	5	3	1	6	4	2			
	7	5	3	1	6	4	2	8		Conflict because $(3, 3) = (8, 8)$
	7	5	3	1	6	4	2			Hence backtrack.
	7	5	3	1	6	4	8			$(6, 5) = (8, 7)$. Not feasible.
	7	5	3	1	6	4				Backtrack.
	7	5	3	1	6	8				
	7	5	3	1	6	8	2			
	7	5	3	1	6	8	2	4		List ends with solution of 8 queen's.

Q.14 Generate at least 3 solutions for 5-queen's problem.

[SPPU : May-12, Marks 8]

Ans. : The solutions are as given below.

	1	2	3	4	5
1	Q				
2			Q		
3					Q
4		Q			
5				Q	

Solution 1

(1, 3, 5, 2, 4)

	1	2	3	4	5
1		Q			
2				Q	
3	Q				
4			Q		
5					Q

Solution 2

(2, 4, 1, 3, 5)

	1	2	3	4	5
1		Q			
2					Q
3			Q		
4	Q				
5				Q	

Solution 3

(2, 5, 3, 1, 4)

4.5 : Graph Coloring Problem

Q.15 Explain graph coloring problem with respect to backtracking.

[SPPU : Dec.-13, 14, Marks 8]

Ans. : Graph coloring is a problem of coloring each vertex in graph in such a way that no two adjacent vertices have same color and yet m -colors are used. This problem is also called **m -coloring problem**. If the degree of given graph is d then we can color it with $d + 1$ colors. The least number of colors needed to color the graph is called its chromatic number. For example : Consider a graph given in Fig. Q.15.1.

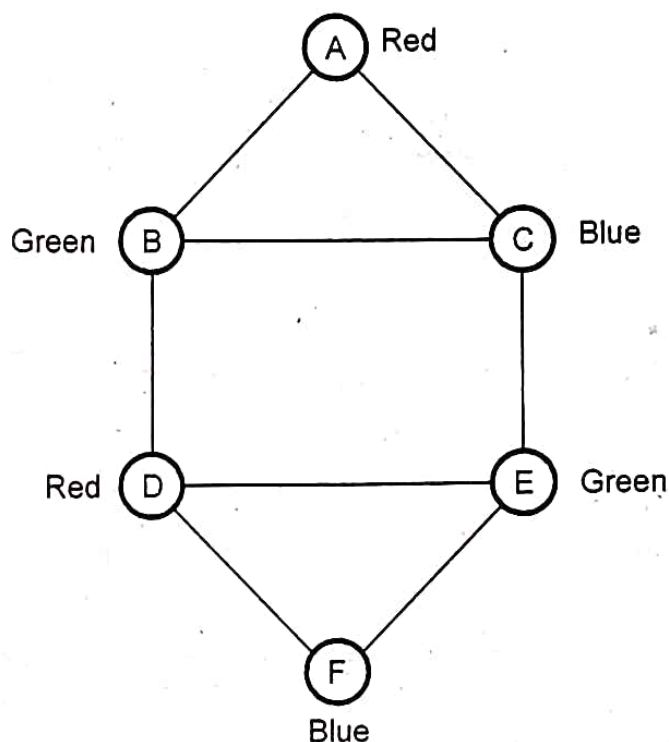
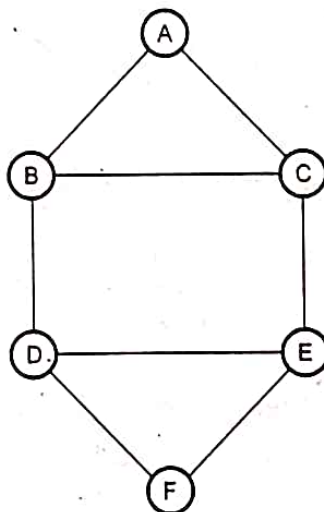


Fig. Q.15.1 Graph and its coloring

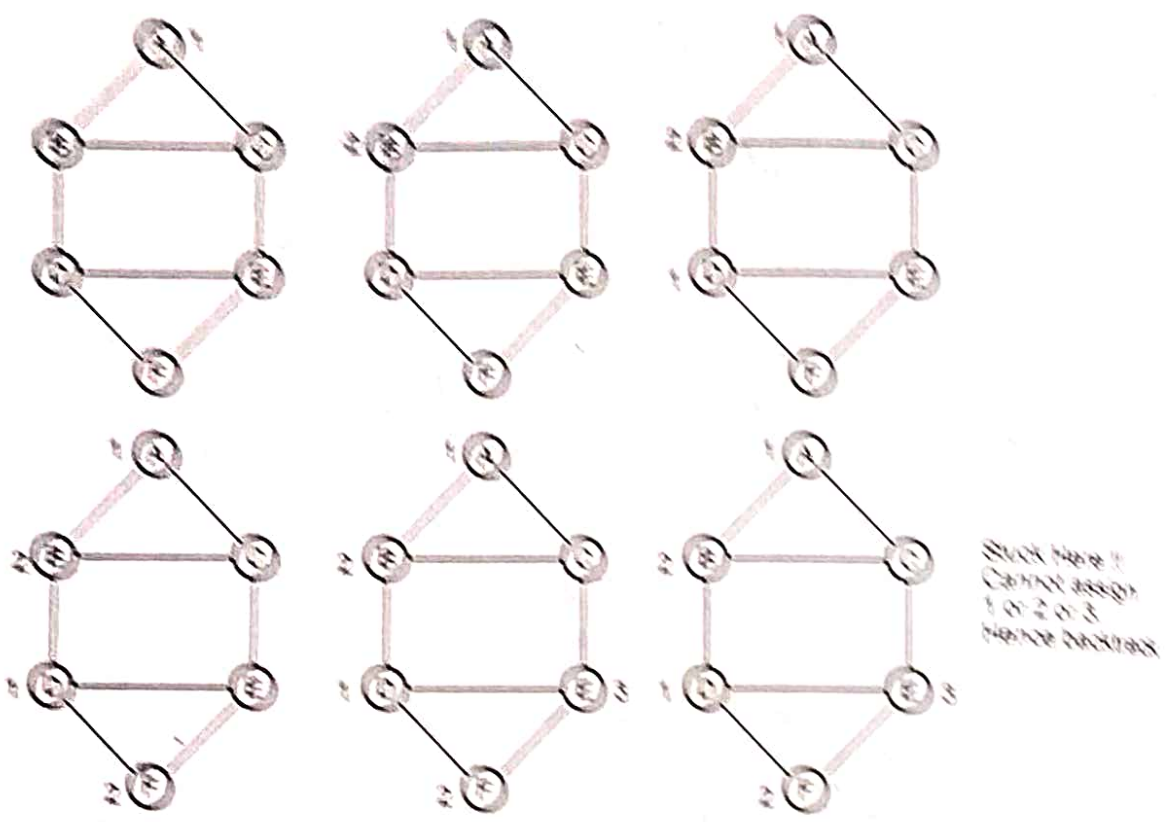
As given in Fig. Q.15.1 we require three colors to color the graph. Hence the chromatic number of given graph is 3. We can use backtracking technique to solve the graph coloring problem as follows -

Step 1 :

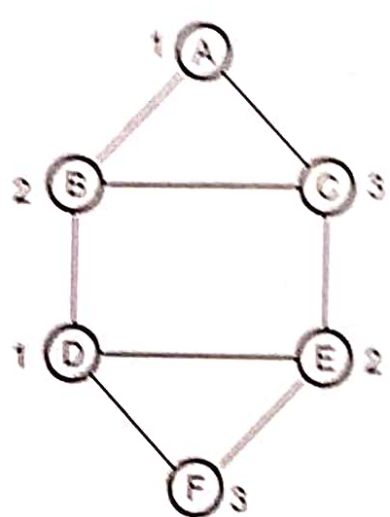
A Graph G consists of vertices from A to F. There are three colors used Red, Green and Blue. We will number them out. That means 1 indicates Red, 2 indicates Green and 3 indicates Blue color.



Step 2 :



Step 3 :



Thus the graph coloring problem is solved. The state-space tree can be drawn for better understanding of graph coloring technique using backtracking approach -

//Each vertex is picked up one by one for coloring
 //x[k] indicates the legal color assigned to the vertex

```
{
  repeat
  { // produces possible colors assigned
```

Gen_Col_Value(k);

Takes $O(nm)$ time

if(x[k] = 0) then

return; //Assignment of new color is not possible.

if(k=n) then // all vertices of the graph are colored

write(x[1:n]); //print color index

else

Gr_color(k+1) // choose next vertex

}until(false);

}

The algorithm used assigning the color is given as below

Algorithm Gen_Col_Value(k)

//x[k] indicates the legal color assigned to the vertex

// If no color exists then x[k] = 0

```
{
```

// repeatedly find different colors

```
repeat
```

```
{
```

x[k] $\leftarrow$ (x[k]+1) mod (m+1) ; //next color index when it is
 //highest index

if(x[k] = 0) then //no new color is remaining

return;

for (j $\leftarrow$ 1 to n) do

```
{
```

// Taking care of having different colors for adjacent
 // vertices by using two conditions i) edge should be
 // present between two vertices

// ii) adjacent vertices should not have same color

if(G[k,j]=0) AND (x[k] $\rightarrow$ x[j]) then

break;


```

}
//if there is no new color remaining
if (j = n+1) then
    return;
}until(false);
}

```

Time complexity :

The Gr_color takes computing time of $\sum_{i=0}^{n-1} m^i$. Hence total computing time for this algorithm is $O(nm^n)$.

Q.17 Construct planar graph for following map. Explain how to find m-coloring of this planar graph by using m-coloring backtracking algorithm. [SPPU : May-11, Marks 10]

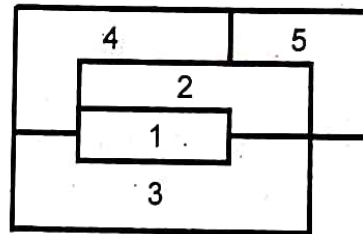


Fig. Q.17.1

Ans. : Steps for graph coloring -

1. Number out each vertex ($V_0, V_1, V_2, \dots, V_{n-1}$)
2. Number out the available colors ($C_0, C_1, C_2, \dots, C_{n-1}$)
3. Start assigning C_i to each V_i . While assigning the colors note that two adjacent vertices should not be colored by the same color. And least number of colors should be used.
4. While assigning the appropriate color, just backtrack and change the color if two adjacent colors are getting the same.

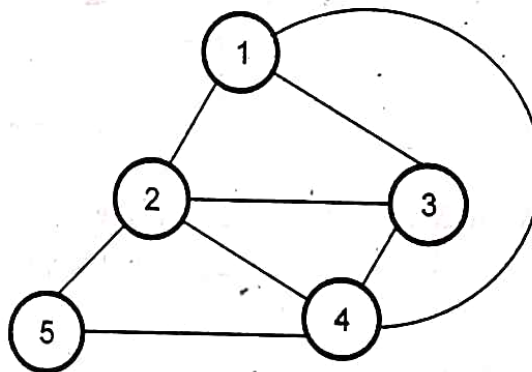


Fig. Q.17.2

4.6 : Sum of Subsets Problem

Q.18 Explain in detail sum of subset problem using backtracking method with suitable example. [SPPU : Aug.-15, In Sem, Marks 4]

Ans. : Problem Statement

Let, $S = \{S_1, \dots, S_n\}$ be a set of n positive integers, then we have to find a subset whose sum is equal to given positive integer d .

It is always convenient to sort the set's elements in ascending order. That is,

$$S_1 \leq S_2 \leq \dots \leq S_n$$

Let us first write a general algorithm for sum of subset problem.

Algorithm

Let, S be a set of elements and d is the expected sum of subsets. Then -

Step 1 : Start with an empty set.

Step 2 : Add to the subset, the next element from the list.

Step 3 : If the subset is having sum d then stop with that subset as solution.

Step 4 : If the subset is not feasible or if we have reached the end of the set then backtrack through the subset until we find the most suitable value.

Step 5 : If the subset is feasible then repeat step 2.

Step 6 : If we have visited all the elements without finding a suitable subset and if backtracking is possible then stop without solution.

Example : Refer Q.19.

Q.19 Analyze sum of subset algorithm on data : $M = 35$ and

i) $W = \{5, 7, 10, 12, 15, 18, 20\}$ ii) $W = \{20, 18, 15, 12, 10, 7, 5\}$

iii) $W = \{15, 7, 20, 5, 18, 10, 12\}$

Are there any discernible differences in the computing time ?

[SPPU : May-11, Marks 10]

Ans. :

Initially Subset = { }	Sum = 0	
5	5	Then add next element
5, 7	$12 < 35$	Add next element

5, 7, 10	$22 < 35$	Add next element
5, 7, 10, 12	$34 < 35$	Add next element
5, 7, 10, 12, 15	$49 > 35$	Sum exceeds $\therefore$ Backtrack
5, 7, 10, 15	$37 > 35$	Backtrack
5, 7, 12	$24 < 35$	Select next element
5, 7, 12, 15	$39 > 35$	Backtrack
5, 10	$15 < 35$	Add next element
5, 10, 12	$27 < 35$	Select next element
5, 10, 12, 15	$42 > 35$	Backtrack
5, 10, 15	$30 < 35$	Select next element
5, 10, 15, 18	$48 > 35$	Backtrack
5, 10, 18	$33 < 35$	Select next element
5, 10, 18, 20	$53 > 35$	Backtrack
5, 10, 20	35	Solution is found

The possible solutions are $\{5, 10, 20\}$, $\{15, 20\}$ and $\{5, 12, 18\}$.

The sequence (ii) $W = \{20, 18, 15, 12, 10, 7, 5\}$ and (iii) $W = \{15, 7, 20, 5, 18, 10, 12\}$ are not in non-decreasing order. The (ii) sequence is in decreasing order and (iii) is totally unsorted.

For sum of subset problem, For efficient findings of solution the weights must be arranged in non-decreasing order. to show how this order is beneficial consider a scenario we have selected the elements $\{5, 7, 10, 12\}$ which sums up 34. Now if we select the next element 15, then this sum will exceed the limit because $49 > 35$.

So we will simply backtrack. It is not even necessary to examine the remaining elements $\{18, 20\}$ of the list. If at all, the list is arranged according to decreasing order of the weights then we have to analyse all the elements of the given list. Similarly if the list is unsorted then each and every element needs to be analysed for required sum. Thus the sequences (ii) and (iii) makes the computing time worst.

4.7 : Principle and Control Abstraction

Q.20 Write short note on - Branch and bound approach.

[SPPU : March-20, Marks 5]

OR Explain branch and bound approach with suitable example. What are general characteristics of branch and bound ?

[SPPU : May-19, Marks 8]

Ans. : • Branch and bound is a general algorithmic method for finding optimal solutions of various optimization problems.

- Branch and bounding method is a general optimization technique that applies where the greedy method and dynamic programming fail. However, it is much slower.
- It often leads to exponential time complexities in the worst case. On the other hand, if applied carefully, it can lead to algorithms that run reasonably fast on average.
- Branch and bound build the state space tree, and find the optimal solution quickly by pruning a few of the branches of the tree that do not satisfy the bound.
- Branch and bound can be useful where some other optimization techniques like greedy or dynamic programming fail.

For Example

Consider the 4-queens problem using branch and bound method. In branch and bound method a bounding function is associated with each node. The node with optimum bounding function becomes an E-node. And children of such node get expanded. The state space tree for the same is as given below -

In Fig. Q.20.1 based on bounding function the nodes will be selected and answer node can be obtained. The numbers that are written outside the node indicates the order in which evaluation of tree takes place. (Refer Fig. Q.20.1 on next page)

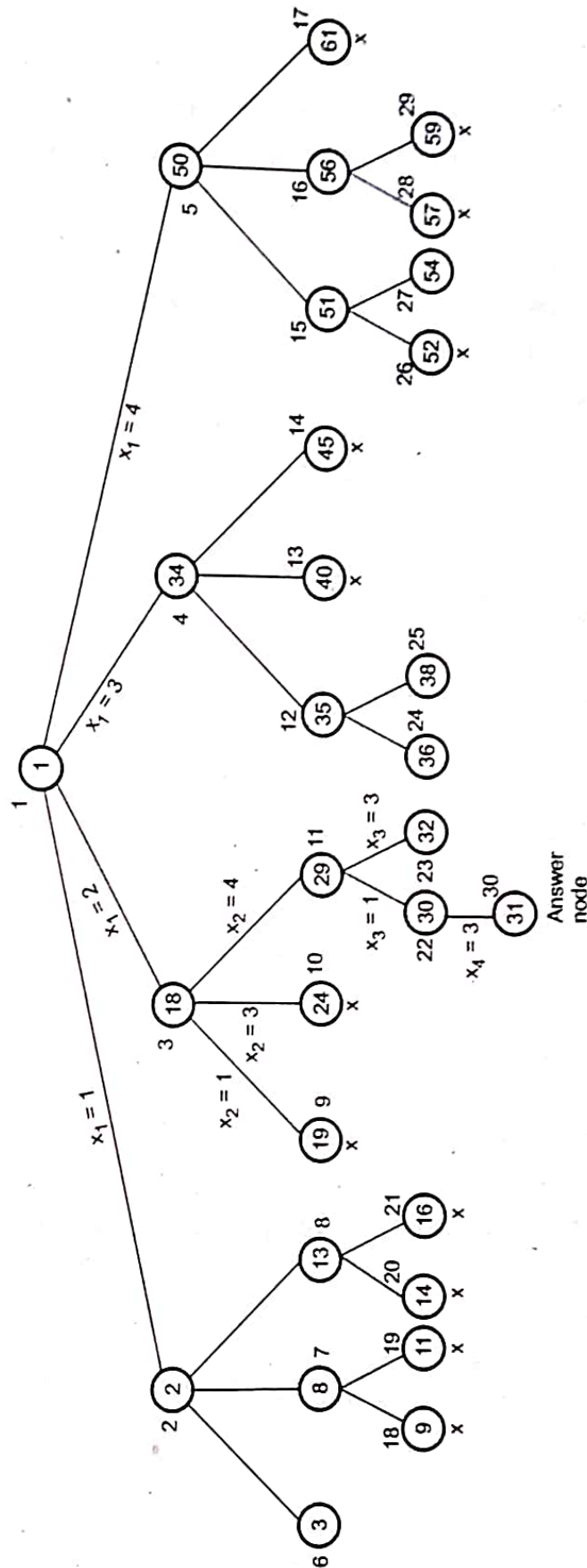


Fig. Q.20.1 Portion of state space tree using 4-queens

4.8 : Strategies- FIFO, LIFO and LC Approaches

Q.21 Explain in detail control abstraction for LC search.

[SPPU : Dec.-08, Marks 8; Dec.-10, May-13, Marks 6]

Ans. : The control abstraction for Least cost search is given as follows.

Algorithm LC_search()

//tr is a state space tree and x be the node in the tr

//E represents the E-node

//initialize the list of live nodes to empty

{

if(tr is answer node) **then**

 {

write(tr);//output the answer node

return;

 }

 E ← tr //set the E-node

repeat

 {

for(each child x of E) **do**

 {

if(x is answer node) **then**

 {

 output the path from x to root;

return;

 }

 //the new live node x will be added in the list of live nodes

 Add_Node(x);

 x → parent ← E;//pointing the path from x to root;

 }

if(no more live nodes) **then**

 {

write("Can not have answer node!!");

return;

 }

// E points to current E-node

```

    E ← Least_cost(); //finds the least cost node to set
    next E-node
  }until(false);
}

```

In above algorithm if x is in tr then $c(x)$ be the minimum cost answer node in tr . The algorithm uses two functions `Least_cost` and `Add_Node()` function to delete or add the live node from the list of live nodes. Using above algorithm we can obtain path from answer node to root. We can use parent to trace the parent of node x . Initially root is the E node. The for loop used in the algorithm examines all the children of E -node for obtaining the answer node. The function `Least_cost()` looks for the next possible E -node.

Q.22 Write the control abstraction for LC-search. Explain how travelling salesperson problem is solved using LCBB.

☞ [SPPU : May-14, Marks 8]

Ans. : Control Abstraction for LC Search : Refer Q.21.

TSP using LCBB : Refer Q.25.

Q.23 What is branch and bound method ? Explain FIFO branch and bound algorithm.

☞ [SPPU : Dec.-14, Marks 8]

Ans. : Branch and bound : Refer Q.20.

FIFO branch and bound Algorithm :

Algorithm FIFOBB()

//tr is a state space tree and x be the node in the tr

//E represents the E-node

//initialize the list of live nodes to empty

{

 if(tr is answer node) then

 {

$u := \min(\text{cost}[tr], (\text{upper}(tr) + E))$

 write(tr); //output the answer node

 return;

 }

$E \leftarrow tr$ //set the E-node

 repeat

 {


```

    for(each child x of E) do
    {
        if(x is answer node) then
        {
            output the path from x to root;
            return;
        } //the new live node x will be added in the list of live
//nodes
        Insert_Q(x);
        x.parent ← E; //pointing the path from x to root;
        if((x is answer node) AND cost(x) < u) then
        {
            u ← min(cost[tr], (upper(tr) + E))
        }
    }
    if(no more live nodes) then
    {
        write("No more Live nodes now");
        write("least Cost is =", u);
        return;
    } // E points to current E-node
    E ← Del_Q(); // deletes the least cost node from Q to set next
                //E-node
    } until(false);
}

```

In above algorithm if x is in tr then $c(x)$ be the minimum cost answer node in tr . The computation of u value is done. We always choose the minimum values of cost of the children generated from the current E node.

The algorithm uses two functions Del_Q and $()$ and $Insert_Q()$ function to delete or add the live node from the list of live nodes stored in queue. Using above algorithm we can obtain path from answer node to root. We can use parent to trace the parent of node x . Initially root is the E node. The for loop used in the algorithm examines all the children of E -node for obtaining the answer node.

4.9 : Concept of Bounding

Q.24 Explain in brief the concept of bounding.

Ans. : • The bounding functions are used to avoid the generation of sub trees that do not contain the answer nodes. In bounding lower bounds and upper bounds are generated at each node.

- A cost function $c^*(x)$ is such that $c^*(x) \leq c(x)$ is used to provide the lower bounds on solution obtained from any node x .
- Let *upper* is an upper bound on cost of minimum-cost solution. In that case, all the live nodes with $c^*(x) > \text{upper}$ can be killed.
- At the start the upper is usually set to ∞ . After generating the children of current E-node, *upper* can be updated by minimum cost answer node. Each time a new answer node can be obtained.

4.10 : Traveling Sales Person Problem

Q.25 Explain dynamic reduction with all steps with respect to travelling salesperson problem.

☞ [SPPU : May-14, Marks 6]

OR Explain the steps of solving travelling sales man problem using branch and bound.

☞ [SPPU : Dec.-14, Marks 8]

OR Explain how travelling salesperson problem is solved using branch and bound method with suitable example.

☞ [SPPU : May-08, Dec.-08, Marks 6 , Dec.-11, 12, Marks 10, Aug.-15, In Sem, Marks 4]

Ans. : Problem Statement : If there are n cities and cost of travelling from any city to any other city is given. Then we have to obtain the cheapest round-trip such that each city is visited exactly once and then returning to starting city, completes the tour.

Typically travelling salesperson problem is represented by weighted graph.

For example : Consider an instance for TSP is given by G as,

$$G = \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

There $n = 5$ nodes. Hence we can draw a state space tree with 5 nodes shown in Fig. Q.25.1.

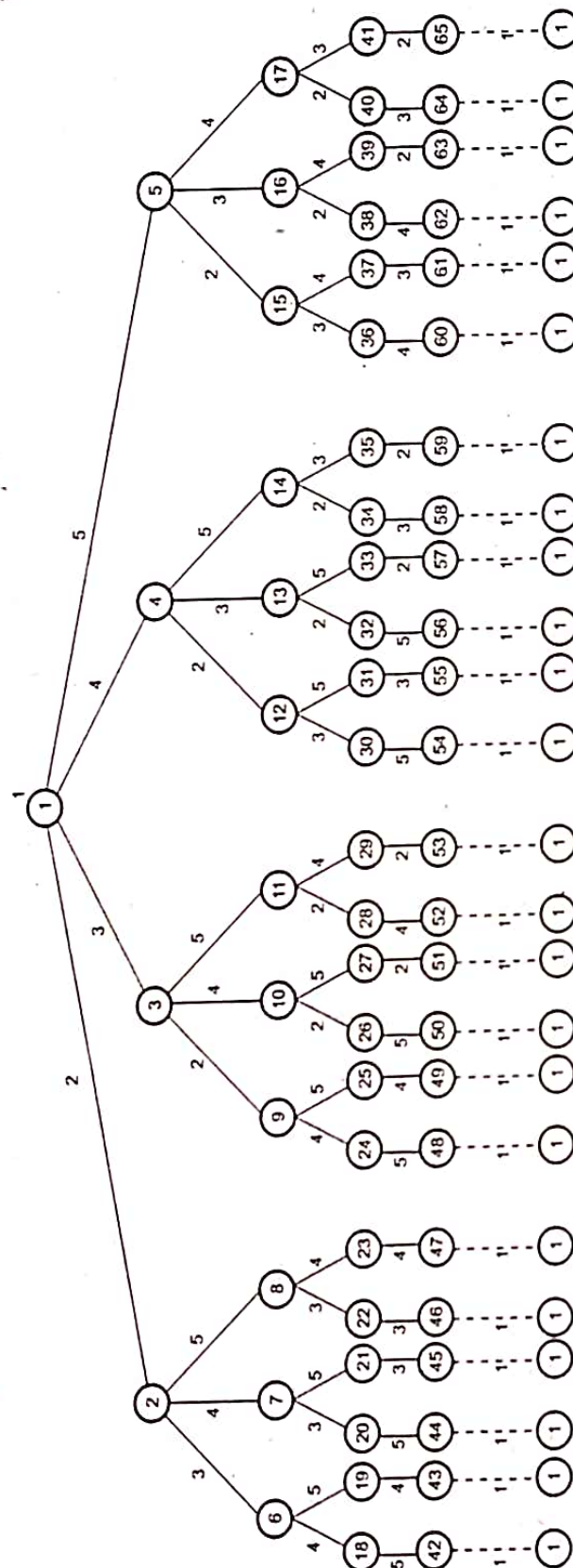


Fig. Q.25.1 State space tree for TSP

Tour(x) is the path that begins at root and reaching to node x in a state space tree and returns to root. In branch and bound strategy cost of each node x is computed. The travelling salesperson problem is solved by choosing the node with optimum cost. Hence,

$$c(x) = \text{cost of tour } (x) \quad \text{where } x \text{ is a leaf node.}$$

The $c^{\wedge}(x)$ is the approximation cost along the path from the root to x.

Row Minimization.

We will reduce the cost of the cost matrix M, by using following formula.

$$\text{Red_Row}(M) = \left[M_{ij} - \min \left\{ M_{ij} \mid 1 \leq j \leq n \right\} \right] \quad \text{where } M_{ij} < \infty$$

For example : Consider the matrix M representing cost between any two cities.

$$M = \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

We will find minimum of each row.

$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix} \quad \begin{matrix} 10 \\ 2 \\ 2 \\ 3 \\ 4 \end{matrix}$$

21 Total reduced cost.

We will subtract the row_minimum value from corresponding row. Hence,

$$\text{Red_Row}(M) = \begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$$

Column Minimization

The formula for column reduction of matrix is,

$$\text{Red_Col}(M) = M_{ji} - \min \left\{ M_{ji} \mid 1 \leq j \leq n \right\}$$

where $M_{ji} < \infty$

Full Reduction

Let M be the cost matrix for travelling salesperson problem for n vertices then M is called reduced if each row and each column consists of either entirely ∞ entries or else contain at least one zero. The full reduction can be achieved by applying both row_reduction and column reduction.

For example : Consider, the matrix M as given below and reduce it fully.

$$M = \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

Red_Row (M) can be obtained as,

$$\begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix} \begin{matrix} \rightarrow 10 \\ \rightarrow 2 \\ \rightarrow 2 \\ \rightarrow 3 \\ \rightarrow 4 \end{matrix}$$

21

$\therefore \text{Cost}(\text{Red_Row}(M)) = 21$

$$\text{Red_Row}(M) = \begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$$

Red_Col(M) be obtained as,

$$\begin{bmatrix} \infty & 10 & 20 & 0 & 1 \\ 13 & \infty & 14 & 2 & 0 \\ 1 & 3 & \infty & 0 & 2 \\ 16 & 3 & 15 & \infty & 0 \\ 12 & 0 & 3 & 12 & \infty \end{bmatrix}$$

↓

↓

1

3

total = 4

$$\therefore \text{Cost (Red_Col (M))} = 4$$

If row or column contains at least one zero ignore corresponding row or column.

$$\text{Red_Col(M)} = \begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

Thus total reduced cost will be

$$= \text{cost (Red_Row (M))} + \text{cost (Red_Col(M))}$$

$$= 21 + 4$$

$$= 25$$

$$\text{Thus } \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 1 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix} \xRightarrow[\text{reduced matrix}]{\text{fully}} \begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

Dynamic Reduction

We obtained the total reduced cost as 25. That means all tours in the original graph have a length at least 25.

Using dynamic reduction we can make the choice of edge $i \rightarrow j$ with optimum cost.

Steps in dynamic reduction technique

1. Draw a state space tree with optimum cost at root node.

2. Obtain the cost of matrix for path $i \rightarrow j$ by making i^{th} row and j^{th} column entries as ∞ . Also set $M[i][j] = \infty$.
3. Cost of corresponding node x with path i, j is optimum cost + reduced cost + $M[i][j]$.
4. Set node with minimum cost as E-node and generate its children. Repeat step 1 to 4 for completing tour with optimum cost.

For example :

$$M = \begin{bmatrix} \infty & 20 & 30 & 10 & 11 \\ 15 & \infty & 16 & 4 & 2 \\ 3 & 5 & \infty & 2 & 4 \\ 19 & 6 & 18 & \infty & 3 \\ 16 & 4 & 7 & 16 & \infty \end{bmatrix}$$

Fully reduced matrix is,

$$\begin{bmatrix} \infty & 10 & 17 & 0 & 1 \\ 12 & \infty & 11 & 2 & 0 \\ 0 & 3 & \infty & 0 & 2 \\ 15 & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & 12 & \infty \end{bmatrix}$$

Optimum cost = $21 + 4 = 25$.

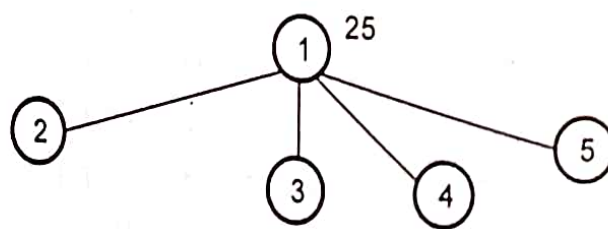


Fig. Q.25.2 Portion of state space tree

Consider path 1, 2. Make 1^{st} row and 2^{nd} column ∞ .

And set $M[2][1] = \infty$.

∞	∞	∞	∞	∞	→ ignore
∞	∞	11	2	0	→ ignore
0	∞	∞	0	2	→ ignore
15	∞	12	∞	0	→ ignore
11	∞	0	12	∞	→ ignore

ignore ignore ignore ignore ignore

Cost of node 2 is

$$\begin{array}{ccccccc}
 25 & + & 0 & + & 10 & & \\
 \uparrow & & \uparrow & & \uparrow & & \\
 \text{optimum} & & \text{reduced} & & \text{old value of} & & \\
 \text{cost} & & \text{cost} & & M[1][2] & &
 \end{array}$$

$$= 35$$

Consider path 1, 3. Make 1st row = ∞ . 3rd column = ∞ and $M[3][1] = \infty$.

∞	∞	∞	∞	∞
12	∞	∞	2	0
∞	3	∞	0	2
15	3	∞	∞	0
11	0	∞	12	∞

↓

11

Cost of node 3 is

$$\begin{array}{ccccccc}
 = & & 25 & + & 11 & + & 17 \\
 & & \uparrow & & \uparrow & & \uparrow \\
 & & \text{optimum cost} & & \text{reduced cost} & & M[1][3] \\
 & & & & & & \\
 & & = & & 53 & &
 \end{array}$$

Consider path 1, 4.

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$

$$\begin{aligned} \text{Cost of node 4 is} &= 25 + 0 + 0 \\ &= 25 \end{aligned}$$

Consider path 1, 5.

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & 2 & \infty \\ 0 & 3 & \infty & 0 & \infty \\ 15 & 3 & 12 & \infty & \infty \\ \infty & 0 & 0 & 12 & \infty \end{bmatrix} \begin{matrix} \rightarrow 2 \\ \rightarrow 3 \end{matrix}$$

$$\begin{aligned} \text{Cost of node 5 is} &= 25 + 5 + 1 \\ &= 31 \end{aligned}$$

Now, as cost of node 4 is optimum, we will set node 4 as E-node and generate its children nodes 6, 7, 8.

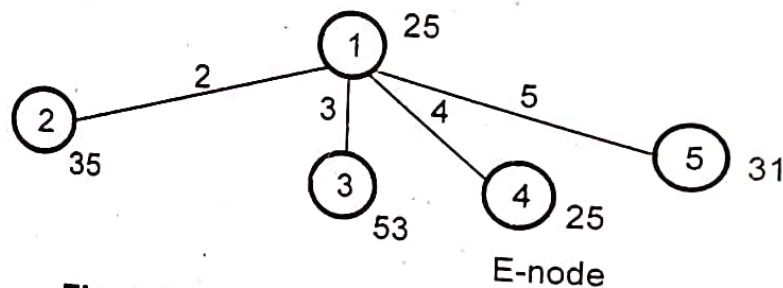


Fig. Q.25.3 Portion of state space tree

Consider path 1, 4, 2 for node 6. Set 1st row, 4th row to ∞ . Set 2nd column to ∞ . Also $M[4][1] = \infty$, $M[2][1] = \infty$.

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix}$$

$$\begin{aligned}
 \text{Cost of node 6} &= 25 + M[4, 2] \\
 &= 25 + 3 \\
 &= 28
 \end{aligned}$$

Consider path 1, 4, 3 for node 7. Set 1st row, 4th row to ∞ . Set 4th column, 3rd column to ∞ . Set $M[4][1] = M[3][1] = \infty$.

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & \infty & \infty & 0 \\ \infty & 3 & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & 0 & \infty & \infty & \infty \end{bmatrix} \rightarrow 2$$

$\uparrow$
 11

$$\begin{aligned}
 \text{Cost of node 7} &= 25 + 13 + M[4][3] \\
 &= 25 + 13 + 12 = 50
 \end{aligned}$$

Consider path 1, 4, 5 for node 8.

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & \infty \\ 0 & 3 & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & 0 & 0 & \infty & \infty \end{bmatrix} \rightarrow 11$$

$$\text{Cost of node 8} = 25 + 11 = 36$$

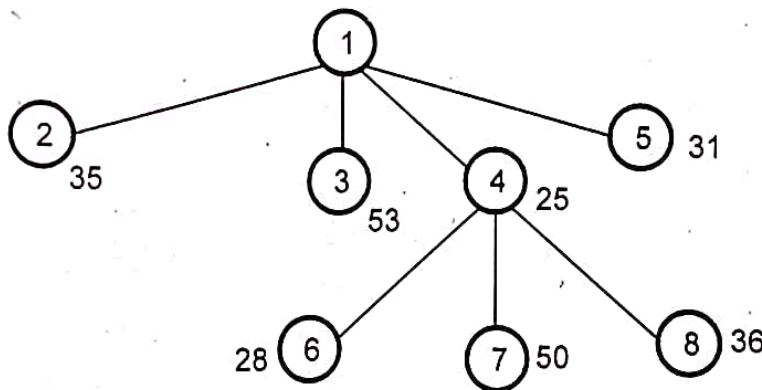


Fig. Q.25.4 Portion of state space tree

Now as cost of node 6 is minimum, node 6 becomes an E-node. Hence generate children for node 6. Node 9 and 10 are children nodes of node 6.

Consider path 1, 4, 2, 3 for node 9. Set 1st row, 4th row, 2nd row to ∞ . Set 4th column, 2nd column and 3rd column to ∞ .

Set $M[4, 1] = M[2, 1] = M[3, 1] = \infty$.

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & \infty & \infty & \infty \end{bmatrix} \rightarrow 2$$

↓

11

Cost of node 9

$$= \begin{matrix} 28 & + & 13 & + & 11 \\ \uparrow & & \uparrow & & \uparrow \\ \text{optimum cost} & & \text{reduced cost} & & M[2][3] \end{matrix}$$

$$= 52$$

Consider path 1, 4, 2, 5 for node 10. Set 1st row, 4th row, 2nd row to ∞ . Set 4th column, 2nd column and 5th column to ∞ . Set $M[4, 1] = M[2, 1] = M[5, 1] = \infty$.

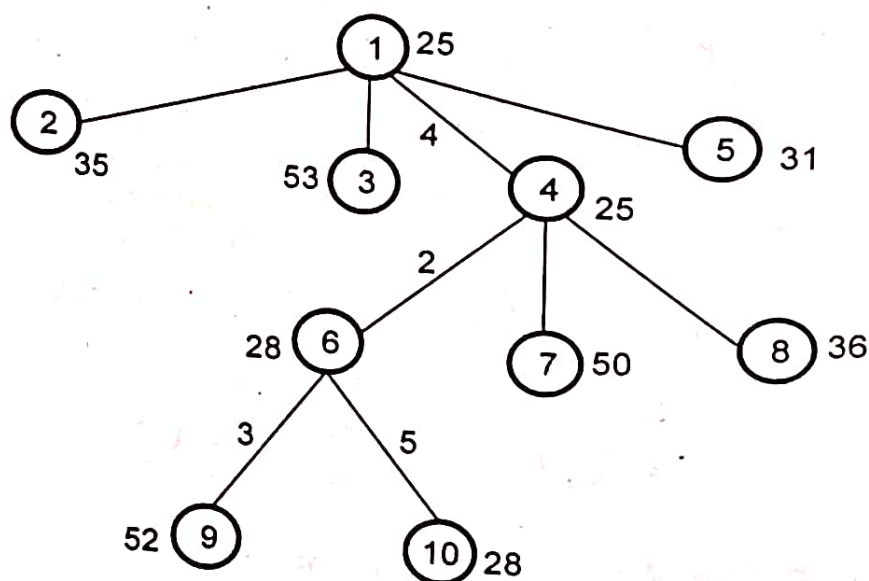


Fig. Q.25.5 Portion of state space tree

$$\begin{bmatrix}
 \infty & \infty & \infty & \infty & \infty \\
 \infty & \infty & \infty & \infty & \infty \\
 0 & \infty & \infty & \infty & \infty \\
 \infty & \infty & \infty & \infty & \infty \\
 \infty & \infty & 0 & \infty & \infty
 \end{bmatrix}$$

$$\begin{aligned}
 \text{Cost of node 10} &= 28 + M[2][5] \\
 &= 28 + 0 \\
 &= 28
 \end{aligned}$$

Node 10 becomes E-node now.

As for node 10 only child being generated is node 11. We set path as 1, 4, 2, 5, 3. To complete the tour we return to 1. Hence the state space tree is,

Hence the optimum cost of the tour is 28.

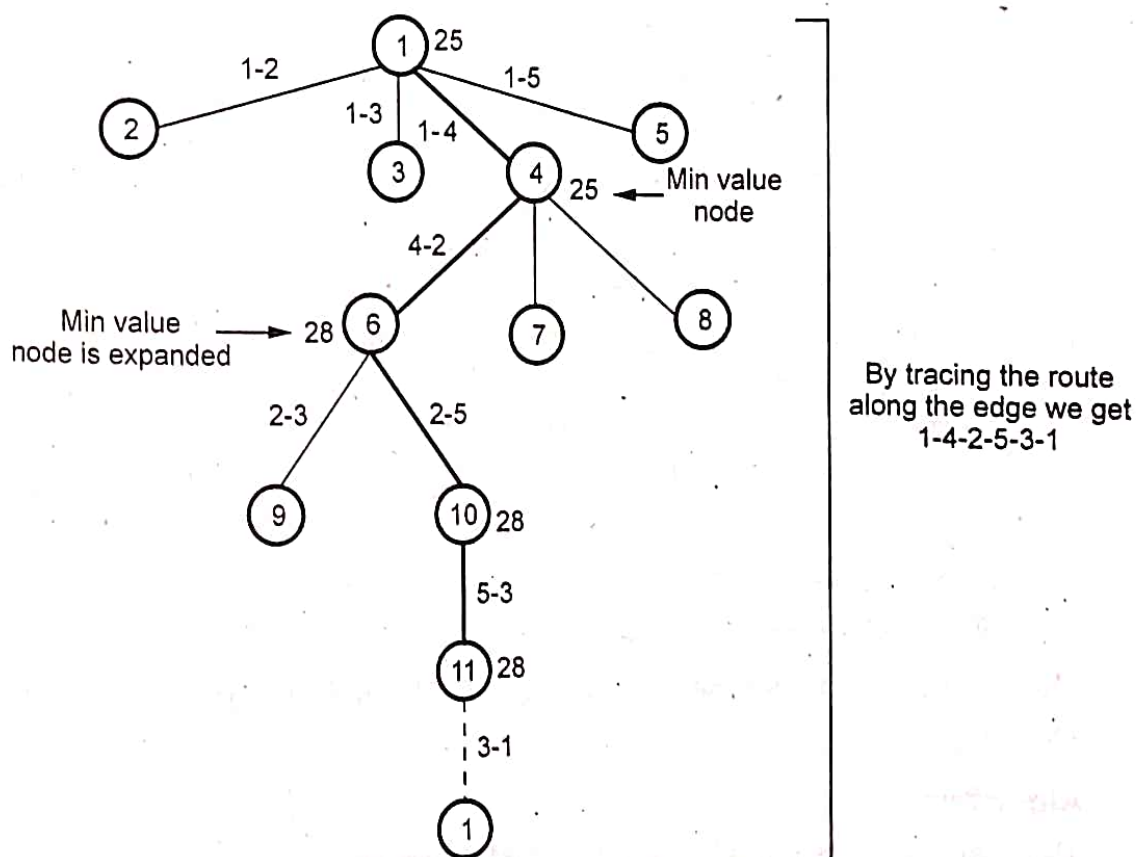


Fig. Q.25.6 State space tree

4.11 : Knapsack Problem

Q.26 Explain the algorithm for implementing the branch bound knapsack problem with suitable example

☞ [SPPU : May-08,09,13, Oct.-16, Marks 10]

Ans. : Problem statement :

The 0/1 Knapsack problem states that - There are 'n' objects given and capacity of Knapsack is 'm'. Then select some objects to fill The knapsack in such a way that it should not exceed the capacity of Knapsack and maximum profit can be earned. The Knapsack problem is a maximization problem. That means we will always seek for maximum $P_i x_i$ (where P_i represents profit of object x_i). We can also get $\sum P_i x_i$ maximum iff $-\sum P_i x_i$ is minimum.

$$\text{Minimize profit} - \sum_{i=1}^n P_i x_i$$

$$\text{subject to } \sum_{i=1}^n W_i x_i$$

$$\text{Such that } \sum W_i x_i \leq m \text{ and}$$

$$x_i = 0 \text{ or } 1 \text{ where } 1 \leq i \leq n.$$

We will design the state space tree and compute $c^{\wedge}(\cdot)$ and $u(\cdot)$ where $c^{\wedge}(x)$ represents the approximate cost used for computing the least cost $c(x)$. Clearly $u(x)$ denotes the **upper bound**. As we know upper bound is used to kill those nodes in the state space tree which can not lead to the answer node.

Let, x be the node at level j . Then we will draw the state space tree for fixed tuple formulation having levels $1 \leq j \leq n+1$.

Then we need to compute $c^{\wedge}(x)$ and $u(x)$. Such that $c^{\wedge}(x) \leq c(x) \leq u(x)$ for every node.

Algorithm

The algorithm for computing $c^{\wedge}(x)$ is as given below.

Algorithm C_Bound(total_profit, total_wt, k)

```
//total_profit denotes the current total profit
//total_wt denotes the current total weight
// k is the index of last removed object
//w[i] represents the weight of object i
//p[i] represents the profit of object i
//m is the weight capacity of knapsack
{
    pt←total_profit;
    wt←total_wt;
    for(i←k+1 to n) do
    {
        wt←wt+w[i];
        if(wt<m) then pt←pt+p[i];
        else return (pt+(1 - (wt-m)/w[i])*p[i]);
    }
    return pt;
}
```

The algorithm for computing $u(x)$ is as given below

Algorithm U_Bound(total_profit,total_wt,k,m)

```
//total_profit denotes the current total profit
//total_wt denotes the current total weight
// k is the index of last removed object
//w[i] represents the weight of object i
//p[i] represents the profit of object i
//m is the weight capacity of knapsack
{
    pt←total_profit;
    wt←total_wt;
    for(i←k+1 to n) do
    {
        if(wt+w[i]<=m) then
        {
```

```

        pt ← pt - p[i];
        wt ← wt + w[i];
    }
}
return pt;
}

```

For example :

Let Number of objects $n = 5$, capacity of knapsack $m = 100$
 Profits = (10, 20, 30, 40, 50), weights = (20, 30, 66, 40, 60)

We will arrange all the items in such a way that $\frac{P_i}{W_i} > \frac{P_{i+1}}{W_{i+1}} > \frac{P_{i+2}}{W_{i+2}} \dots$

The arrangement will be

Item	P_i	W_i
4	40	40
5	50	60
2	20	30
1	10	20
3	30	66

- Here we will select item 4, item 5. Now if we select item 2 then total weight $> m$. Hence by selecting item 4 and item 5 the upper bound for profit is obtained i.e.

$40 + 50 = 90 \therefore$ we will write $ub = -90$ for root node, of BB tree. As by choosing item 4 and item 5 the total capacity of knapsack is fulfilled. Hence

$$c^{\wedge} = ub = -90$$

- Now if we don't select item 4. Then consider selection of item 5 and item 2. Then upper bound for profit is $ub = -70$. By this selection the total weight $= 60 + 30 = 90$. Now we need only weight 10 to reach to maximum capacity of knapsack.
 $\therefore$ Just consider next item for selection i.e. item 1 but we need its fractional weight

$$\therefore c^{\wedge} = \text{ub} + \text{Fractional profit of next item}$$

$$= (70) + \left(\frac{10}{20} * 10 \right)$$

$$= 75$$

$$\therefore c^{\wedge} = -75$$

• The partial state space tree will be -

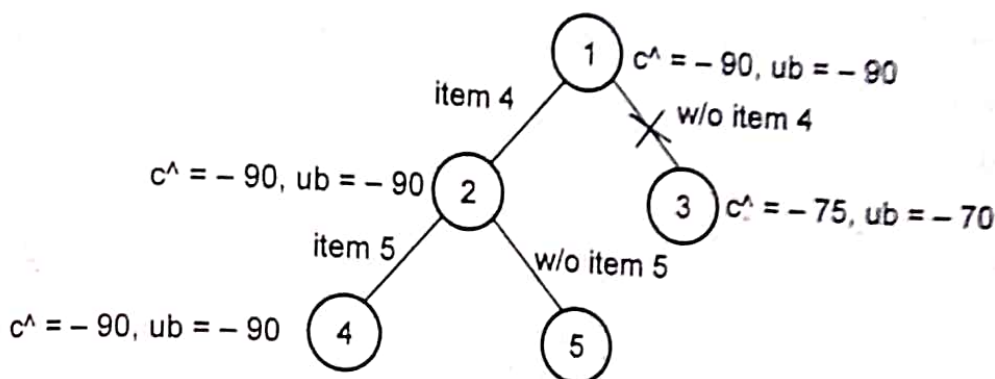


Fig. Q.26.1

We will choose node 2 because it is having less upper bound than node 3. The node 4 and node 5 has the same $C^{\wedge}$ and ub as that of node 2 because it considers selection of item 4 and item 5.

Now consider node 5 i.e. select item 4 and item 2 $\text{ub} = -(40 + 20) = -60$. The next remaining weight of item 1 is 20 which can be added as a whole.

$$\therefore C^{\wedge} = -(\text{ub} + \text{weight of item 1})$$

$$= -(60 + 10)$$

$$\therefore C^{\wedge} = -70$$

$$\therefore \text{At node 5 } C^{\wedge} = -70, \text{ub} = -60$$

But as node 4 is having lesser upper bound than node 5. Select node 4.

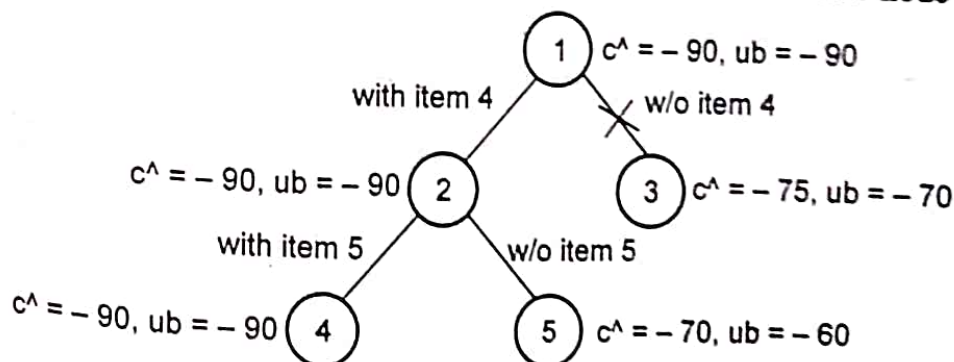


Fig. Q.26.2

Now can not further select item 2, item 1 or item 3 because by selecting item 4 and item 5 the knapsack has reached to its capacity.

Hence solution = {item 4, item 5} with maximum profit = 90.

4.12 : Difference between Backtracking and Branch and Bound Techniques

Q.27 Give the difference between backtracking and branch and bound techniques

Ans. :

Sr. No.	Backtracking	Branch and Bound
1.	The solution for backtracking is traced using depth first search.	In this technique it is not necessary to use depth first search for obtaining the solution, we can use breadth first search algorithm.
2.	Typically the decision problems are solved using backtracking.	Typically the optimization problems are solved using branch and bound.
3.	While finding the solution, the bad choices can be made.	It proceeds on better solutions. There cannot be a bad solution.
4.	The state space tree is searched until the solution is obtained.	The state space tree needs to be searched completely as there may be the chances of being an optimum solution anywhere in the state space tree.
5.	Applications - M-Coloring, eight queens problem.	Applications - Job sequencing, Traveling Salesperson Problem.

END...

Unit V

5

Amortized Analysis

5.1 : Amortized Analysis

Q.1 Explain in brief the concept of amortized analysis ?

Ans. : An amortized analysis means finding average running time per operation over a **worst case sequence** of operations. An amortized analysis indicates that average cost of a single operation is small if average of sequence of operations is obtained. This is true even if any one operation is expensive within the sequence. An amortized analysis guarantees the **time per operation over worst case performance**.

Q.2 What is the difference between amortized analysis and average case analysis ?

Ans. : There is a difference between average case analysis and amortized analysis. In average case analysis we are averaging over all possible inputs and in amortized analysis we are averaging over a sequence of operations. An amortized analysis assumes worst-case input.

Q.3 Enlist the commonly used techniques used in amortized analysis

Ans. : The commonly used techniques in amortized analysis are -

- (1) Aggregate Analysis
- (2) Accounting Method
- (3) Potential Method

Q.4 Explain in brief amortized analysis. Find the amortized cost with respect to stack operations. [SPPU : Dec.-07, Marks 10, May 18, Marks 8]

OR Name and explain in two or three sentences three popular methods to arrive at amortized costs for the operations.

[SPPU : May-08, Marks 6]

Ans. : Amortized Analysis : Refer Q.1 and Q.2.

Amortized Cost with Respect to Stack Operations :

(1) Aggregate Analysis

Aggregate analysis is a kind of analysis in which the analysis is made over sequence of n operations and these operations actually take worst case time $T(n)$. In worst case an amortized cost per operation is $T(n)/n$.

1. To implement 'push (item)' we need :

$s[\text{top}] = \text{item};$

$\text{top} ++;$

2. To implement $\text{item} = \text{pop} ()$ we need :

$\text{top} --;$

$\text{item} = s[\text{top}];$

Thus **push (item)** and **pop ()** are the operations each running in $O(1)$ time. This also means that cost of each operation is $O(1)$ and for n sequences of operations the total execution time will be $\Theta(n)$. Now if we write some function for performing **multiple pop** operations then k top objects will be removed from the stack.

Algorithm multiple _ pop (st, k)

{

// **Problem Description** : This algorithm is for

// removing k top objects repeatedly using stack.

while ((! stempty (st)) **AND** $k \neq 0$)

{

pop (st)

$k \leftarrow k - 1$

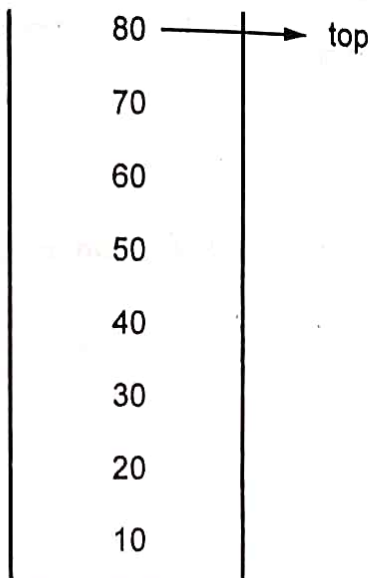
} // end if while

} // end of algorithm

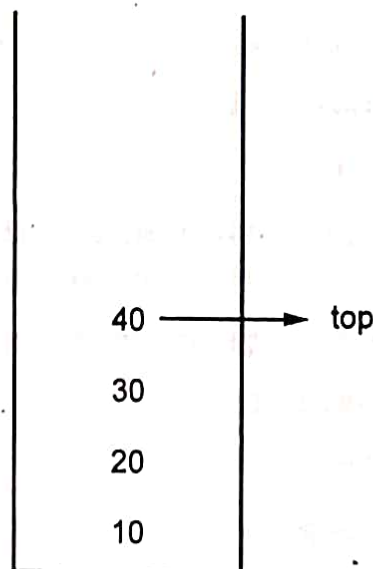
While stack do not gets empty the desired elements can be removed.

For example :

If we want to remove 4 elements from the stack then-



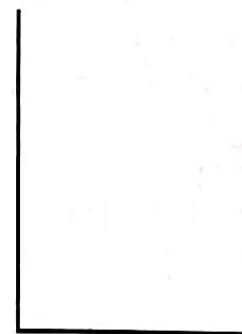
Initially stack
has 8 element



Using `multiple_pop`
4 elements can be
removed

That means for the size 8 of the stack we can perform at the most 8 pop operations.

- From above given sequence of execution we can state that worst case cost for **multiple_pop** is $O(n)$ when the size of stack is at the most n . Hence a sequence of n operations costs $O(n^2)$ because we can have $O(n)$ **multiple_pop** operations costing $O(n)$ each. The cost $O(n^2)$ is correct but not tight.
- We can broadly state that each object can be popped off once. That also means : Total number of pop = Total number of push.
- The aggregate analysis suggests to define amortized cost to be **average cost**.
- Hence average cost in amortized analysis is nothing but **average cost** of an operation = $O(n)n$



Empty stack after
removing 4 elements

(2) Accounting Method

The counting method is based on the charges that are assigned to each operation.

- Thus in accounting method we can say

$$\text{Amortized cost} = \text{actual cost} + \text{credits (either deposited or used up)}$$

Let us first assign some charges to each stack operation.

Push (item) : = 1

Pop () : 1

multiple_pop: min (st, k) where st is stack size and k is number of multiple pops. These cost are the **actual costs**.

Let us now assign amortized costs as -

push (item) : 2

pop () : 0

multiple_pop : 0

Note that **multiple_pop** operation has variable actual cost and 0 amortized cost. Now consider some sequence of operations starting from empty stack. For now, if we push some object onto the stack then associated with it \$2 as amortized cost. We then can use \$1 to pay the actual cost of push and left with \$1 as credit. If we go on pushing the objects the credits will get accumulated each time. Now, while popping the object each credit will be used to prepay the cost of pop operation. Thus by charging **more** on **push** operation we need **not** have to **charge** anything on **pop** operation. The same is true for **multiple_pop** operation. This also ensures that credits are always **non negative**.

Hence total amortized cost for n push (item), pop, multiple_pop is $O(n)$.
Hence average amortized cost for each operation is $O(n) / n = 1$

(3) Potential Method

This method is similar to the accounting method in which the concept of "prepaid" is used.

- In this method there will be some potential "energy" or "potential" which can be used to pay for future operations.
- The **working** of potential method is as follows -
- Let, D_0 be the initial data structure. Hence for n operations $D_0, D_1, D_2, \dots, D_n$ will be the data structure. Let $c_1, c_2, \dots, c_n$ denotes the actual cost.

- Let Φ is a potential function which is a real number. $\Phi(D_i)$ is called potential of D_i .
- Let, c'_i be the **amortized cost** of i^{th} operation

$$c'_i = c_i + \underbrace{\Phi(D_i) - \Phi(D_{i-1})}_{\text{Potential change}}$$

Actual cost

$$\sum_{i=1}^n c'_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) \quad \text{and}$$

$$\sum_{i=1}^n c'_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$$

- If $\Phi(D_n) \geq \Phi(D_0)$, then amortized cost is an **upper bound** of actual cost.
- For any sequence of operation we do not know the exact number of operations. Hence it is required to have $\Phi(D_i) \geq \Phi(D_0)$ for any value of i .
- Let $\Phi(D_0) = 0$. So $\Phi(D_i) \geq 0$ for all i .
- If **potential change** is **positive** i.e. if $\Phi(D_i) - \Phi(D_{i-1}) > 0$ then c'_i is overcharge and potential of data structure increases.
- If potential change is negative i.e. if $\Phi(D_i) - \Phi(D_{i-1}) < 0$ then c'_i is an undercharge i.e. discharge the potential to pay actual cost.

The number of objects in the stack is the potential for the stack. So

$\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ i.e. non negative value.

We can now obtain amortized cost of each stack operation as follows -

Push operation

$$\begin{aligned} \text{Potential change} &= \Phi(D_i) - \Phi(D_{i-1}) \\ &= (s + 1) - s \\ &= 1 \end{aligned}$$

$s + 1$ means inserting one item to existing stack of size s

Amortized cost = actual cost + potential change

$$\begin{aligned} &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + 1 \end{aligned}$$

$$c'_i = 2$$

Pop operation

$$\begin{aligned}
 \text{Potential change} &= \Phi(D_i) - \Phi(D_{i-1}) \\
 &= (s-1) - s \\
 &= -1
 \end{aligned}$$

$s-1$ means
popping off one
item from stack
of size s

$$\begin{aligned}
 \text{Amortized cost} &= \text{actual cost} + \text{potential change} \\
 &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\
 &= 1 + (-1)
 \end{aligned}$$

$$c'_i = 0$$

Multiple_pop

$$\begin{aligned}
 \text{Potential change} &= \Phi(D_i) - \Phi(D_{i-1}) \\
 &= k'
 \end{aligned}$$

where $k' = \min(st, k)$ with k denotes number of pop and st denotes stack size.

$$\begin{aligned}
 \text{Amortized cost} &= \text{Actual cost} + \text{Potential change} \\
 &= c_i + \Phi(D_i) - \Phi(D_{i-1}) = k' + k' \\
 c'_i &= 0
 \end{aligned}$$

Thus **amortized cost** of each operation is $O(1)$ and total amortized cost of n operations is $O(n)$.

The **worst case cost** of n operations is $O(n)$.

Q.5 What is amortized analysis ? Explain aggregate and accounting techniques with example.

[SPPU : May-19, Marks 8]

Ans. : Refer Q.4.

5.2 : Time-Space Trade-off

Q.6 Explain the concept of time-space trade-off with suitable example.

Ans. : • **Definition :** Time space tradeoff is basically a situation where either a space efficiency (memory utilization) can be achieved at the cost of time or a time efficiency (performance efficiency) can be achieved at the cost of memory.

• **Example :** Consider the programs like compilers in which symbol table is used to handle the variables and constants. Now if entire symbol table is stored in the program then the time required for searching or

storing the variable in the symbol table will be reduced but memory requirement will be more. On the other hand, if we do not store the symbol table in the program and simply compute the table entries then memory will be reduced but the processing time will be more.

Q.7 Write and explain the issues in time-space trade-off.

Ans. :

1. When time and space compete with each other an **efficient algorithm** with **space-efficient data structure** has to be used.

For example in sparse Matrix (the matrix which contains very less number of values, and almost all the matrix contains zero).

2. **Data compression** techniques should be applied in order to reduce the size of data used in computation.

5.3 : Introduction to Tractable and Non-tractable Problems

Q.8 What is tractable and non-tractable problem ?

Ans. : • **Tractable problem** : A problem that is solvable by a polynomial-time algorithm. The upper bound is polynomial.

• **Examples of tractable problems are -**

- i) Searching an unordered list ii) Searching an ordered list
- iii) Sorting a list iv) Multiplication of integers
- v) Finding a minimum spanning tree in a graph.

• **Intractable problem** : A problem that cannot be solved by a polynomial-time algorithm. The lower bound is exponential.

• **Examples of intractable problems -**

- i) Traveling Salesman Problem ii) Knapsack Problem

5.4 : Introduction to Randomized Algorithms

Q.9 Write a short note on randomized algorithm.

[SPPU : Dec.-15, End Sem, Marks 8, Dec.-19, Marks 4]

Ans. : A random number generator or randomizer makes use of randomized algorithms. In randomized algorithms the decision is always taken based current output.

For a randomized algorithm -

- i) Execution time may vary from run to run.
- ii) For different inputs there may be different outcomes on each execution.
- iii) For same inputs there may be different outcomes on each execution.

There are two types of randomized algorithms.

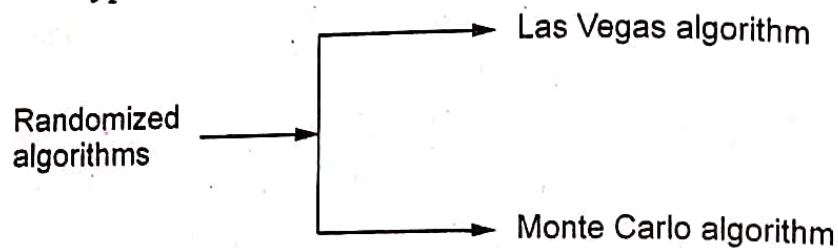


Fig. Q.9.1

Las Vegas algorithms : Are the type of randomized algorithms which produce same outcome on each execution for same input.

Monte Carlo algorithms : Are the type of randomized algorithms which produce different outcomes on each execution for same input.

Randomized algorithm for Quick Sort : Refer Q.9.

Advantages and Disadvantages :

There are two major advantages of randomized algorithms.

1. These algorithms are simple to implement.
2. These algorithms are many times efficient than traditional algorithms.

However randomized algorithms may have some drawbacks -

1. The small degree of error may be dangerous for some applications.
2. It is not always possible to obtain better results using randomized algorithm.

Q.10 Explain randomized algorithm for quick sort.

[SPPU : Dec.-18, Marks 8]

Ans. : The randomized algorithm for quick sort is as follows -

RANDOMIZED-PARTITION(A, p,r)

- 1 $i \leftarrow \text{RANDOM}(p,r)$
- 2 exchange $A[r] \leftrightarrow A[i]$
- 3 return PARTITION(A, p,r)

The new quicksort calls RANDOMIZED-PARTITION in place of PARTITION :

RANDOMIZED-QUICKSORT(A, p,r)

- 1 if $p < r$
- 2 then $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p,r)$
- 3 **RANDOMIZED-QUICKSORT**(A, p, $q - 1$)
- 4 **RANDOMIZED-QUICKSORT**(A, $q + 1,r$)

5.5 : Introduction to Approximate Algorithms

Q.11 Explain the concept of approximation algorithm.

[SPPU : Dec.-19, Marks 4]

Ans. : • The approximation algorithms are algorithms used to find approximate solutions to optimization problems.

- Approximation algorithms are often associated with NP-hard problems because it is very difficult to get an efficient polynomial time exact algorithm for solving NP-hard problems.
- That approximate algorithms are mainly useful for solving those problems where exact polynomial algorithms are known but running such algorithm is too expensive because of their sizes of data sets.
- The philosophy behind approximation algorithm is *find 'good' solution fast*. That means you won't get the exact solution but you will get the approximate solution quickly.
- For approximation algorithm we start with inaccurate data, hence optimization may be as good as optimal solution. Hence approximation algorithms are based on heuristics (i.e. proceeding to solution by trial and error).
- A **heuristics** can be defined as a collection of common sense rules defined from experience.
- **For example :** In travelling salesperson problem going to next nearest city, or in knapsack problem start with highest value and minimum weight item.
- **Accuracy Ratio**
It is always necessary to know the accuracy of approximation to the actual optimal solution. Hence accuracy ratio is defined as

$$r(s_a) = \frac{f(s_a)}{f(s^*)}$$

where s_a denotes approximate solution, $r(s_a)$ denotes accuracy ratio, $f(s_a)$ is a value of objective function for solution given by approximation algorithm, $f(s^*)$ is a value of objective function. Generally $r(s_a) \geq 1$. When $r(s_a)$ reaches close to 1 then it is a better approximate solution.

• Performance Ratio

The best upper bound on accuracy ratio taken over all instances of the problem is called performance ratio. It is denoted by R_A . By knowing performance ratio one can judge **quality** of approximation algorithm. The approximation algorithms with R_A value nearer to 1 is supposed to be a better approximation algorithm.

Q.12 Explain the concept of randomized algorithm and approximation algorithm in brief with example.

[SPPU : May-16, End Sem, May-19, Marks 8]

Ans. : Refer Q.9 and Q.11.

5.6 : Embedded Algorithms

Q.13 Write a short note on - embedded systems.

Ans. : Definition of embedded system : The embedded system is a computer system with a dedicated function within a larger mechanical or electrical system.

The embedded system typically does not look like a computer, often there is no keyboard or monitor or mouse. But like a computer it has a processor, memory and software. The word embedded means a permanent part within a big system. For example - Smart Card, Washing Machines, Microwave Ovens, answering machine, Mobile Phones, digital Cameras, Vending Machines and so on.

Characteristics of embedded systems

1. **Single Functional :** The embedded systems are usually designed to perform dedicated functions.
2. **Tightly constrained :** Embedded systems are computing systems with tightly coupled hardware and software integration.
3. **Part of large systems :** The word embedded means the system is an integral part of large system.

3. **Reactive and Real time** : These systems are real time systems and respond to the events and triggers. These systems are generally time bounded. Modern embedded systems have user friendly interface.
4. **Programmable** : The embedded systems contain at least one programmable unit (micro-controller).

Q.14 What is embedded system ? Explain embedded system scheduling.

[SPPU : Dec.-15, Marks 9, May-19, Marks 8]

Ans. : Embedded System : Refer Q.13.

Embedded system scheduling : Fixed priority algorithm

- The fixed-priority algorithm assigns all priorities to the tasks at design time. Those priorities remain constant for the lifetime of the task.
- The implementation of this algorithm is as follows -

Step 1 : Initially the tasks reside in the **sleep queue**.

Step 2 : When the tasks are ready for execution they enter in the **ready queue**. One ready queue for each distinct priority is maintained.

Step 3 : The highest priority task will be executed first.

Fig. Q.14.1 (a) illustrates this implementation.

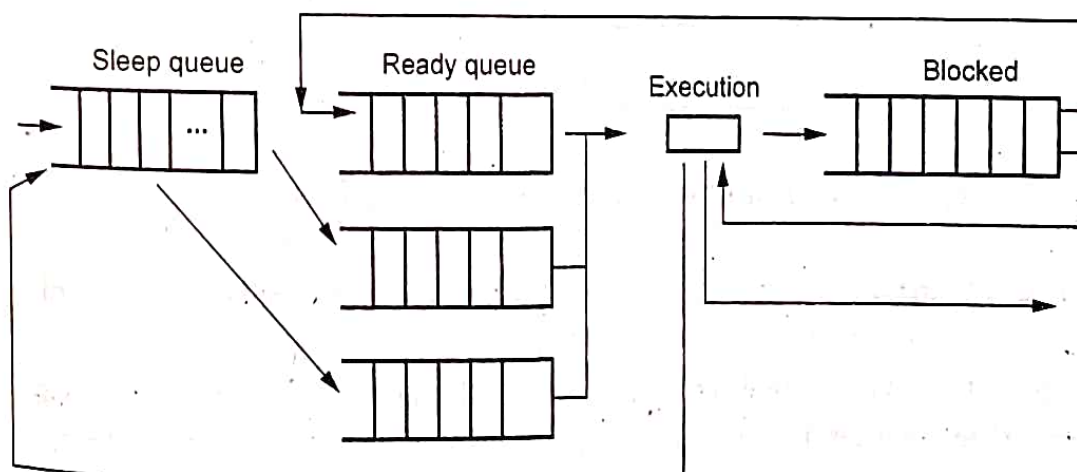


Fig. Q.14.1 (a) Priority queues for fixed priority algorithm

- The **advantage** is this algorithm is it is simple to execute.

Dynamic priority algorithm

- All the scheduling decisions such as assignment of priorities to the task is at run time. This scheduling algorithm takes **task deadlines** into consideration which are used for deciding the priorities. The mutual exclusion and synchronization is enforced in dynamic priority algorithm.
- The implementation of this algorithm is as follows -

Step 1 : Initially the tasks reside in the **sleep queue**.

Step 2 : When the tasks are ready for execution they enter in the ready queue. One ready queue for each distinct priority is maintained.

Step 3 : For each highest priority task, check the deadlines of highest priority queue and then these tasks will be executed.

Fig. Q.14.1 (b) illustrates this implementation

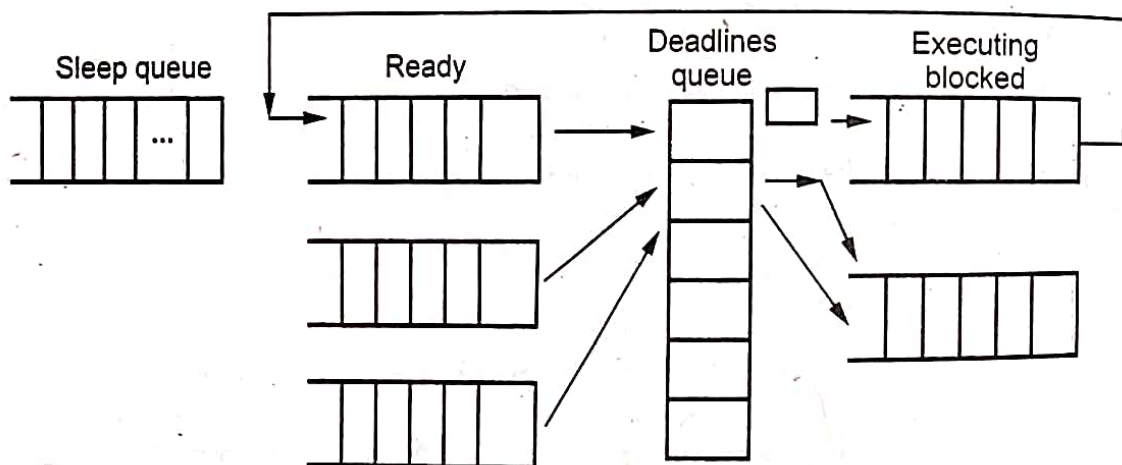


Fig. Q.14.1 (b) Priority queues for dynamics priority algorithm

- The advantage of this algorithm is that it is **flexible** and it is used only when the actual resources are claimed.

Q.15 What is embedded system ? Explain embedded sorting algorithm [SPPU : Dec.-15, End Sem, Marks 9, May-18,19, Dec.-19, Marks 8]

Ans. : Embedded System : Refer Q.13.

Embedded system sorting : Insertion sort : Generally the insertion sort maintains a zone of sorted elements. If any unsorted element is obtained then it is compared with the elements in the sorted zone and then it is inserted at the proper position in the sorted zone. This process is

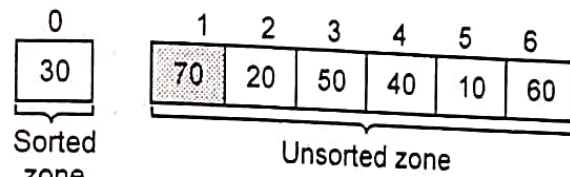
continued until we get the complete sorted list. Hence the name of this method is insertion sort.

For example

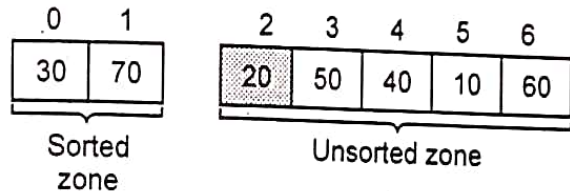
Consider a list of elements as,

0	1	2	3	4	5	6
30	70	20	50	40	10	60

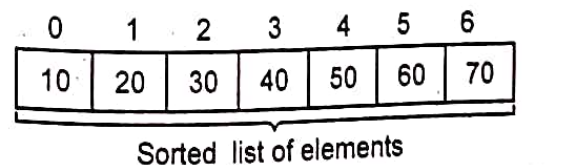
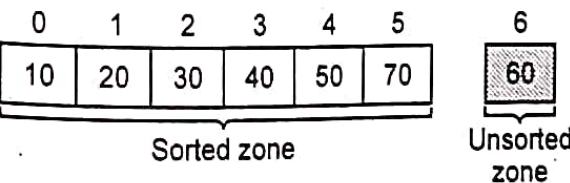
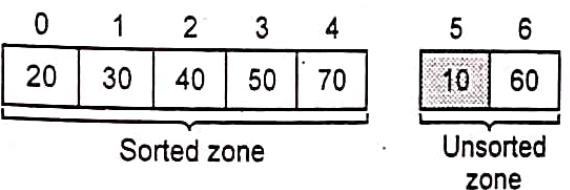
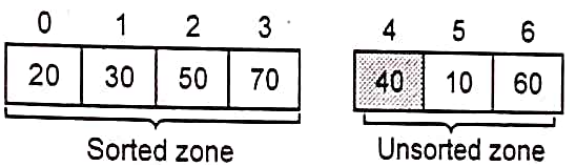
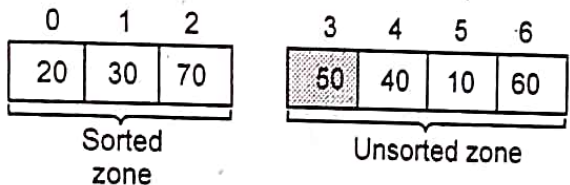
The process starts with first element



Compare 70 with 30 and insert it at its position



Compare 20 with the elements in sorted zone and insert it in that zone at appropriate position



Algorithm

Although it is very natural to implement insertion using recursive (top down) algorithm but it is very efficient to implement it using bottom up (iterative) approach.

Algorithm Insert_sort($A[0 \dots n-1]$)

//Problem Description: This algorithm is for sorting the

//elements using insertion sort

//Input: An array of n elements

//Output: Sorted array $A[0 \dots n-1]$ in ascending order

for $i \leftarrow 1$ to $n-1$ **do**

{

temp $\leftarrow A[i]$ //mark $A[i]$ th element

$j \leftarrow i-1$ //set j at previous element of $A[i]$

while ($j \geq 0$) **AND** ($A[j] > \text{temp}$) **do**

{

//comparing all the previous elements of $A[i]$ with

// $A[i]$. If any greater element is found then insert

//it at proper position

$A[j+1] \leftarrow A[j]$

$j \leftarrow j-1$

}

$A[j+1] \leftarrow \text{temp}$ //copy $A[i]$ element at $A[j+1]$

}

END...

Unit VI

6

Multithreaded and Distributed Algorithms

6.1 : Multithreaded Algorithms

Q.1 What is multithreaded algorithm?

Ans. : • **Multithreaded algorithms** are parallel algorithms which can run on parallel computers that permits multiple instructions to execute concurrently. This exhibits the model of **dynamic multithreaded algorithms**.

- **Static threading** is a kind of programming in which the processors on which the thread is executed is managed explicitly. On the other hand, during **dynamic multithreading** the programmers specify the parallelism. The **concurrency platform** manages parallel computing resources.
- **Dynamic multithreading** allows programmers to specify parallelism in applications without worrying about communication protocols, load balancing, and so on.

Q.2 Define performance measure of multithreaded algorithms. Write multithreaded algorithm for fibonacci series and explain it with suitable example.

Ans. :

1. Work : The work is defined as the total time to execute the entire computation on one processor. In other words, the work is the sum of the times taken by each of the strands.

Work Law (T_1)

Let,

T_1 is the total time to execute algorithm on one processor.

T_p is the total time taken to execute algorithm on P processors.

As $T_p \geq T_1$ we get the work law as

$$T_p \geq T_1/P$$

2. Span (T_∞) : The span is the longest time to execute the strands along any path in the directed acyclic graph.

Span Law

Let, T_∞ is the total time to execute an algorithm on an infinite number of processors.

T_∞ is called the span because it corresponds to the longest time to execute the strands along any path in the computation dag.

Hence the span law states that a P-processor ideal parallel computer cannot run faster than one with an infinite number of processors :

$$T_p \geq T_\infty$$

Performance Measure of Fibonacci(4) Execution

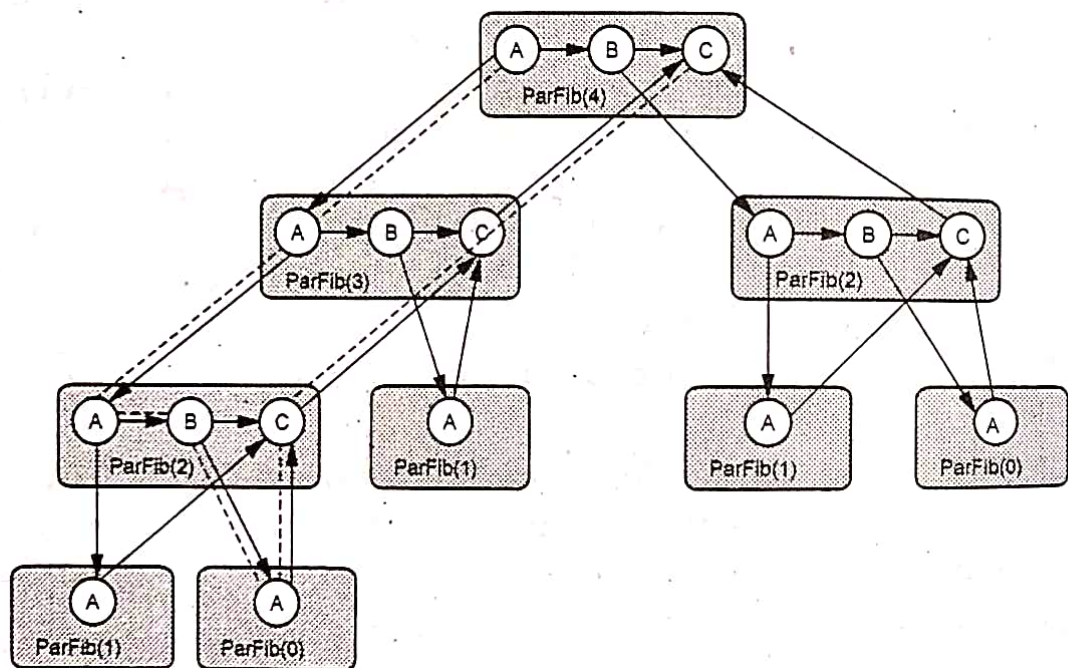


Fig. Q.2.1 Performance measure of Fib (4)

Here there are in all 17

vertices = 17 threads

8 vertices on longest path

Assuming unit time for each thread we get

Work = 17 time units

Span = 8 time units

Speedup

The ratio T_1 / T_P defines how much speedup we get with P processors.

By the work law,

$$T_P \geq T_1 / P,$$

so $T_1 / T_P \geq P$

That means one cannot have any more speedup than the number of processors.

When the speedup $T_1 / T_P = \Theta(P)$ we have **linear speedup**, and when $T_1 / T_P = P$ we have **perfect linear speedup**.

Parallelism

The ratio T_1 / T_∞ of the work to the span gives the potential parallelism of the computation.

$$\text{Parallelism} = \text{Work/Span}$$

It can be interpreted in three ways :

- (i) **Ratio** : The average amount of work that can be performed for each step of parallel execution time.
- (ii) **Upper bound** : the maximum possible speedup that can be achieved on any number of processors.
- (iii) **Limit** : The limit on the possibility of attaining perfect linear speedup. Once the number of processors exceeds the parallelism, the computation cannot possibly achieve perfect linear speedup. The more processors we use beyond parallelism, the less perfect the speedup.

Parallel Slackness

The parallel slackness for multithreaded computation executed on ideal parallel computers with P processors is the ratio of parallelism by P

It is given as -

$$\begin{aligned} & (T_1 / T_\infty) / P \\ & = T_1 / (P \cdot T_\infty) \end{aligned}$$

If slackness is less than 1 then perfect linear speedup is not possible: you have more processors than you can make use of. If slackness is greater than 1, then the work per processor is the limiting constraint and a scheduler can strive for linear speedup by distributing the work across more processors.

Q.3 Explain multithreaded algorithms. How to analyze multithreaded algorithms ? What is race condition, parallel loops ?

[SPPU : May-19, Marks 9]

Ans. : Multithreaded algorithm : Refer Q.1.

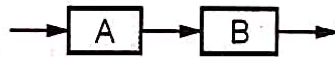
Analysis of Multithreaded algorithm : Analysing work for parallel algorithms is very simple. Just ignore the parallel constructs and analyze the serial algorithm. For example, the work of ParFib(n) is $T_1(n) = T(n) = \Theta(Fn)$.

Analyzing span requires additional efforts.

For example -

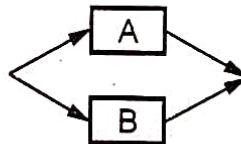
Consider that there are two processors A and B. Following figure shows what will be the work and span when these processors are connected in series and in parallel

In series simple sum of



$$\text{Work : } T_1(A \cup B) = T_1(A) + T_1(B)$$

$$\text{Span : } T_\infty(A \cup B) = T_\infty(A) + T_\infty(B)$$



$$\text{Work : } T_1(A \cup B) = T_1(A) + T_1(B)$$

$$\text{Span : } T_\infty(A \cup B) = \max(T_\infty(A), T_\infty(B))$$

Race Condition : A multithreaded algorithm is deterministic if and only if it does the same thing on the same input, no matter how the instructions are scheduled.

- A multithreaded algorithm is nondeterministic if its behavior might vary from run to run.
- The multithreaded algorithms are supposed to be deterministic but they are actually nondeterministic because they contain **determinacy race** situation.

- **Determinacy race** is a situation which occurs when two logically parallel instructions access the same memory location and at least one of the instructions performs a write. This condition is called race condition.

- For example - Consider following algorithm, that brings the race condition

RaceCondition()

a = 0

parallel for i = 1 to 2 do

 //parallel execution of instructions

 a = a+1 //Writing at the same memory location.

print a

- There are three operations performed in above algorithm and those are
 - 1) Read a from memory into one of the processor's registers
 - 2) Increment the value of the register
 - 3) Write the value in the register back into a in memory
- This can be illustrated by following Fig. Q.3.1.

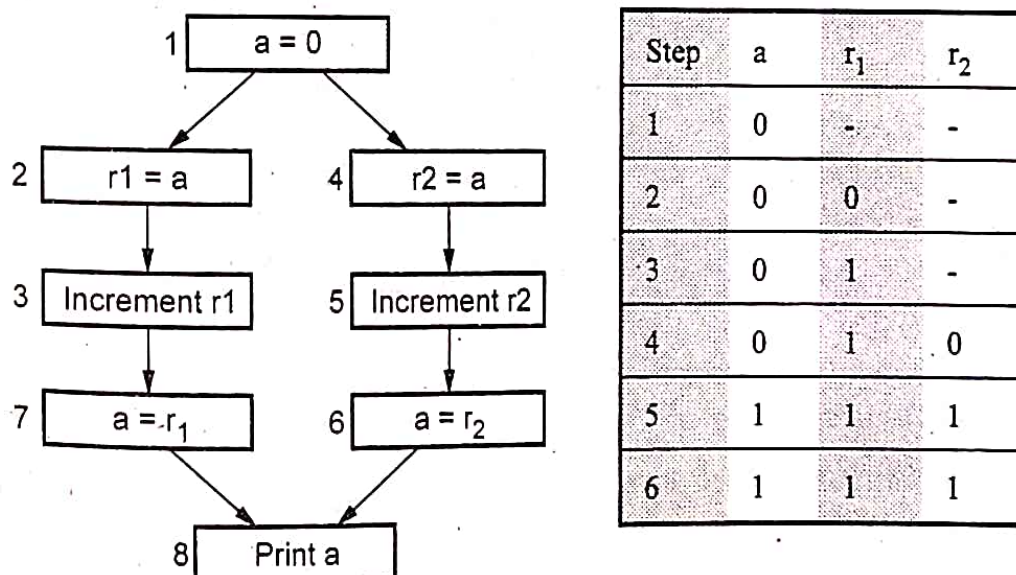


Fig. Q.3.1 Determinacy race

- In above case, the execution order (1, 2, 3, 4, 5, 7, 6, 8) or (1, 2, 4, 3, 5, 7, 6, 8) causes bugs.

Parallel Loops : In parallel algorithms we can use the keyword **parallel** with **for** construct instead of **spawn**.

Suppose we want to multiply an $n \times n$ matrix $A = (a_{ij})$ by an n -vector $x = (x_j)$. This yields an n -vector $y = (y_i)$ where :

$$y_i = \sum_{j=1}^n a_{ij} x_j$$

for $i = 1, 2, \dots, n$.

The following algorithm does this in parallel :

MatrixVector(A,x)

```

1  n = A . rows
2  let y be a new vector of length n
3  parallel for i = 1 to n
4  yi = 0
5  parallel for i = 1 to n
6  for j = 1 to n
7  yi = yi + aij xj
8  return y
```

Row multiplication

The **parallel for** keywords indicate that each iteration of the loop can be executed concurrently.

6.2 : Problem Solving using Multithreaded Algorithms

Q.4 Give pseudo code for multithreaded matrix multiplication. Analyze the same.

[SPPU : May-19, Marks 9]

Ans. : Matrix multiplication operation is possible by applying divide and conquer strategy.

To multiply two $n \times n$ matrices, we perform 8 matrix multiplications of $(n/2) \times (n/2)$ matrices and one addition of $n \times n$ matrices.

Following formula is used to multiply the two matrices using divide and conquer strategy

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

Then we can write the product of two matrices as -

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$= \begin{pmatrix} A_{11}B_{11} & A_{11}B_{12} \\ A_{21}B_{11} & A_{21}B_{12} \end{pmatrix} + \begin{pmatrix} A_{12}B_{21} & A_{12}B_{22} \\ A_{22}B_{21} & A_{22}B_{22} \end{pmatrix}$$

Using the above formula, the parallel matrix multiplication algorithm is as given below

PMatMulRec(Matrix C, Matrix A, Matrix B)

- 1 $n = A.rows$
- 2 if $n == 1$
- 3 $c_{11} = a_{11}b_{11}$
- 4 else let T be a new $n * n$ matrix
- 5 partition A, B, C, and T into $n/2 * n/2$ submatrices
 $A_{11}, A_{12}, A_{21}, A_{22}, B_{11}, B_{12}, B_{21}, B_{22}, C_{11}, C_{12}, C_{21}, C_{22};$
 and $T_{11}, T_{12}, T_{21}, T_{22}$ respectively
- 6 spawn PMatMulRec(C_{11}, A_{11}, B_{11})
- 7 spawn PMatMulRec (C_{12}, A_{11}, B_{12})
- 8 spawn PMatMulRec(C_{21}, A_{21}, B_{11})
- 9 spawn PMatMulRec(C_{22}, A_{21}, B_{12})
- 10 spawn PMatMulRec(T_{11}, A_{12}, B_{21})
- 11 spawn PMatMulRec(T_{12}, A_{12}, B_{22})
- 12 spawn PMatMulRec(T_{21}, A_{22}, B_{21})
- 13 PMatMulRec (T_{22}, A_{22}, B_{22})
- 14 sync
- 15 parallel for $i = 1$ to n
- 16 parallel for $j = 1$ to n
- 17 $c_{ij} = c_{ij} + t_{ij}$

Analysis

In above algorithm, there are 8 multiplications performed recursively of $n/2 * n/2$ matrices. Each recursion is of $\Theta(1)$ time. Hence in all we get 8 $M_1(n/2)$. Similarly there are two parallel for loops for performing matrix addition. Hence this addition costs $\Theta(n^2)$

The recursion for work is then given by

$$M_1(n) = 8M_1(n/2) + \Theta(n^2)$$

$$M_1(n) = \Theta(n^3)$$

...(1)

Similarly to determine the span $M_\infty(n)$, the eight parallel recursive calls that execute are of same size, the maximum span for any recursive call is just the span of any one. Similarly the span for two parallel for loops that perform addition is of $\Theta(\log n)$.

Hence recurrence for span is given be

$$M_\infty(n) = M_\infty(n/2) + \Theta(\log n)$$

$$M_\infty(n) = \Theta(\log^2 n)$$

...(2)

From equations (1) and (2) the parallelism can be

$$\begin{aligned} \text{Parallelism} &= M_1(n)/M_\infty(n) \\ &= \Theta(n^3)/\Theta(\log^2 n) \end{aligned}$$

Thus parallelism is very high. High parallelism means we can dispatch more tasks simultaneously to different resources.

Q.5 Write and explain, multi-threaded merge sort algorithm.

[SPPU : May-18, Dec.-19, Marks 9, Dec.-18, Marks 8]

Ans. : • For implementing the multithreaded merge sort, we have to make the merge operation parallel.

• The divide and conquer strategy will imply that the sorting is done to break the list into four lists. Two of them will be merged to form **head** and other two will be merged to form the **tail**.

• To find the four lists, following steps are applied -

(1) Choose the longer list to be the first list, $T[a_1..c_1]$ in the Fig. Q.5.1.

(2) Find the middle element (median) of the first list (x at b_1).

(3) Use binary search to find the position (b_2) of this element if it were to be inserted in the second list $T[a_2..c_2]$.

(4) Recursively merge.

(a) The first list up to just before the median $T[a_1..b_1-1]$ and the second list up to the insertion point $T[a_2..b_2-1]$

(b) The first list from just after the median $T[b_1+1..c_1]$ and the second list after the insertion point $T[b_2..c_2]$

- Assemble the results with the median element placed between them, as shown below.

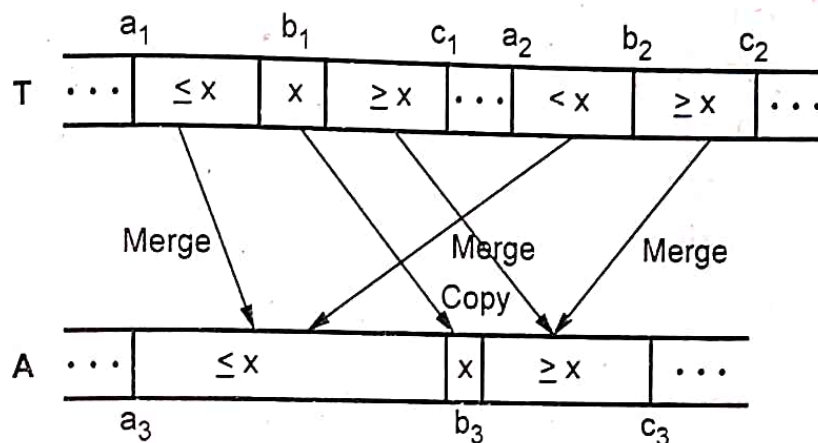


Fig. Q.5.1

Thus the parallel merge sort algorithm $\text{PMergeSort}(\text{Array } A, a, c, \text{Array } B, m)$ which sorts the elements in array $A[a \dots c]$ and stores them in $B[m \dots m+c-a]$

The complete multithreaded merge sort algorithm is as follows -

PMergeSort(Array A, a, c, Array B, m)

{

$n = c - a + 1$

if $n == 1$

$B[m] = A[a]$

else let $T[1 \dots n]$ be a new array

$b = \text{floor}((a+c)/2)$

$b' = b - a + 1$

spawn **PMergeSort**(A, a, b, T, 1)

PMergeSort(A, b+1, c, T, b'+1)

sync

PMerge(T, 1, b', b'+1, n, B, m)

}

In above algorithm we have called **PMerge** i.e. parallel merge function. The algorithm for this is as given below -

PMerge(T, a1, c1, a2, c2, A, a3)

```

{
    n1 = c1 - a1 + 1
    n2 = c2 - a2 + 1
    if n1 < n2 // Here n1 >= n2
        exchange a1 with a2
        exchange c1 with c2
        exchange n1 with n2
    if n1 == 0
        return
    else
        b1 = floor((a1 + c1)/2)
        b2 = BinarySearch(T[b1], T, a2, c2)
        b3 = a3 + (b1 - a1) + (b2 - a2)
        A[b3] = T[b1]
        spawn PMerge(T, a1, b1-1, a2, b2-1, A, a3)
        PMerge(T, b1+1, c1, b2, c2, A, b3+1)
    sync
}

```

6.3 : Distributed Algorithms

Q.6 Explain the concept of distributed algorithm.

Ans. : Definition : Distributed algorithms are those algorithms that are supposed to work in distributed network or on multiprocessor. The distributed network is created by interconnected processors.

- Distributed algorithms are a sub-type of parallel algorithm, typically executed concurrently, with separate parts of the algorithm being run simultaneously on independent processors, and having limited information about what the other parts of the algorithm are doing.

Q.7 What is distributed algorithm ? Explain distributed breadth first algorithm with example.

[SPPU : May-18,19, Dec.-19, Marks 9, Dec.-18, Marks 7]

Ans. : Distributed Algorithm : Refer Q.6.

Distributed Breadth First Search :

The distributed Breadth First Search algorithm minimizes the maximum communication time from the process at distinguished node to all other processes in the network.

For the Breadth First Search (BFS) we assume that the network is strongly connected and there is a distinguished source node i_0 . The algorithm is supposed to output the structure of breadth first search directed tree of network graph rooted at i_0 .

The output should appear in distributed fashion - that means each process other than i_0 should have parent component that gets set to indicate the node which is its parent in the tree.

The algorithm is as follows -

Step 1 : At any point during execution, there is set of processes that is marked initially as i_0 .

Step 2 : Process i_0 sends out **search message** at round 1 to all its outgoing neighbors.

Step 3 : From round 2 to n if unmarked process receives **search message**, it marks itself and chooses one of the processes from which it receives search message as its **parent**. Then it sends the **search message** to all its outgoing neighbors..

Step 4 : Finally a BFS tree is obtained.

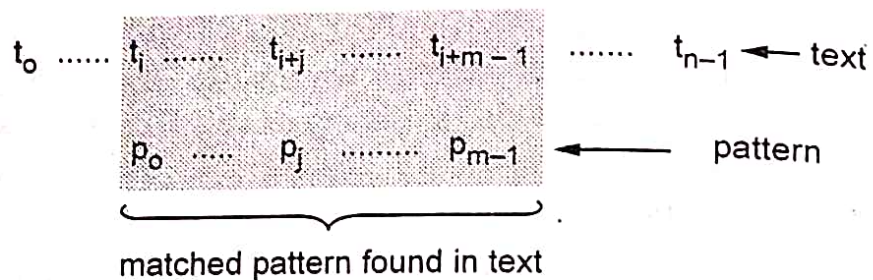
6.4 : String Matching

Q.8 What are string matching algorithms ?

Ans. : • String matching algorithms are normally used in **text processing**.

- Normally text processing is done in compilation of program. In software design or in system design also text processing is a vital activity. And while processing the text, string matching is an important activity which is needed, most of the time.
- String matching means finding one or more generally all the occurrences of a string in the text. These occurrences are called as **pattern**. Hence sometimes string matching algorithms are also called as **pattern matching** algorithms.

Let, Text T is denoted by $t_0 \dots t_{n-1}$ and
pattern P is denoted by $p_0 \dots p_{m-1}$



Q.9 Write naive string matching algorithm. What is the time complexity of algorithm ? Explain complexity of algorithm with example.

[SPPU : Dec.-18, Marks 9]

Ans. : Algorithm :

Algorithm Naive ($T[1\dots n]$, $P[1\dots m]$)

```
{
// Problem Description : This algorithm finds
// the string matching using Naive method.
// Input : The array text T and pattern P
for (s ← 0 to n-m) do
{
    while ( $P[1\dots m] = T[s+1\dots s+m]$ ) then
        print ("pattern finding with shift", S);
}
} // end of algorithm
```

Time complexity

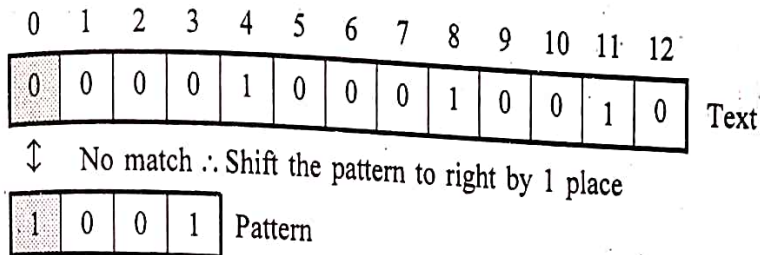
In the given example, if a match is not found then shift the pattern to right by 1 position i.e. almost always we are shifting the pattern to the right. The worst case occurs when we have to make all the m comparisons. In above implementation, we can see that for loop is executed at the most $(n - m + 1)$ times and inner while loop executes for m times. Hence running time in worst case is $O((n - m + 1) m)$ i.e. $O(nm)$. For a typical word search in natural language the average case efficiency is $\Theta(n)$.

Example : Refer Q.10.

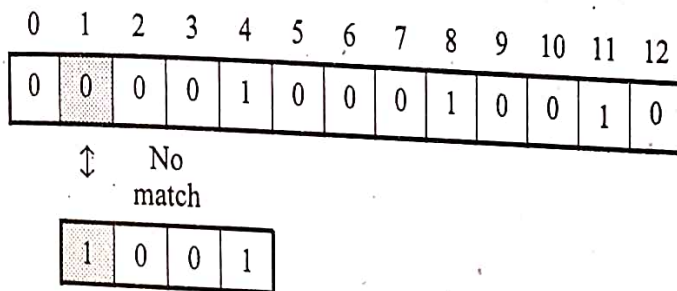
Q.10 Show the comparisons the Naive-String matcher makes for the pattern $P = \{1001\}$ in the text $T = \{0000100010010\}$

Ans. : This is the simplest method of string matching. The match for the pattern can be obtained as follows -

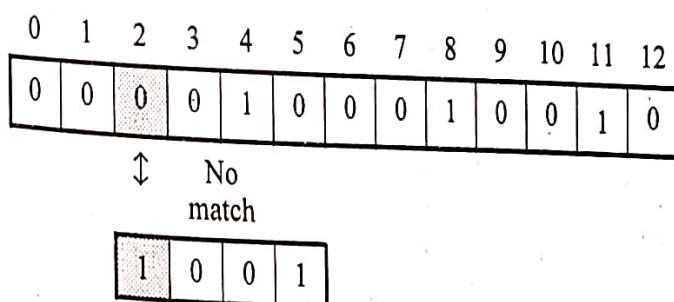
Step 1 :



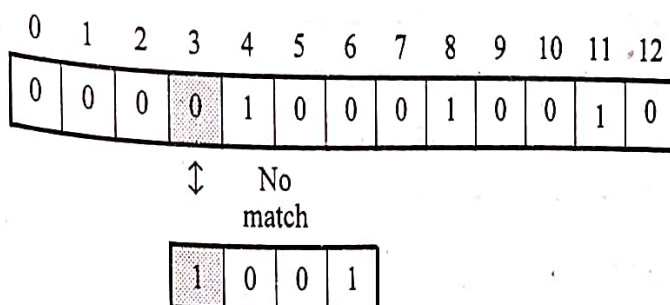
Step 2 :



Step 3 :



Step 4 :



Step 5 :

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	1	0	0	0	1	0	0	1	0

⇕ ⇕ ⇕ Only three characters match. Hence shift the pattern to right

1	0	0	1
---	---	---	---

Step 6 :

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	1	0	0	0	1	0	0	1	0

⇕ No match

1	0	0	1
---	---	---	---

Step 7 :

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	1	0	0	0	1	0	0	1	0

⇕ No match

1	0	0	1
---	---	---	---

Step 8 :

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	1	0	0	0	1	0	0	1	0

⇕ No match

1	0	0	1
---	---	---	---

Step 9 :

0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	1	0	0	0	1	0	0	1	0

								↑	↑	↑	↑	
								1	0	0	1	

Thus match is found at index 8 in text.

Q.11 Write and explain Rabin-Karp string matching algorithm.

[SPPU : Dec.-18, Marks 10]

Ans. : Algorithm -

Algorithm RabinKarp ($T[1.....n]$, $P[1.....m]$, d , q)

{

// Problem Description : This algorithm is

// for matching the pattern using Rabin Karp method

$h \leftarrow \text{pow}(d, m-1) \bmod q$ ← high order digit position

$p \leftarrow 0$

$t_0 \leftarrow 0$

for ($i \leftarrow 1$ to m)

{

$p \leftarrow (dp + P[i]) \bmod q$
 $t_0 \leftarrow (dt_0 + T[i] \bmod q)$ } $\Theta(m)$ time

}

for ($s \leftarrow 0$ to $n - m$) // with shift value each time

{

// incremented by one we

// go on matching pattern

if ($p = t_s$) then // against text.

{

if ($T[1.....m] = T[s+1s+m]$) then

write(" pattern found with shift," s)

}

if ($s < n - m$)

Yet end of text is not reached.

$$t_{s+1} \leftarrow (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$$

} // end of for

} // end of algorithm

Computing

t_{s+1}

The matching time required by above algorithm is $\Theta((n - m + 1)m)$. Hence worst case running time of this algorithm is $\Theta((n - m + 1)m)$.

Example - Refer Q.12.

Q.12 Using the Rabin - Karp algorithm for text $T = 3141592653589793$ find for pattern $P = 26$. With modulo $q = 11$

Ans. :

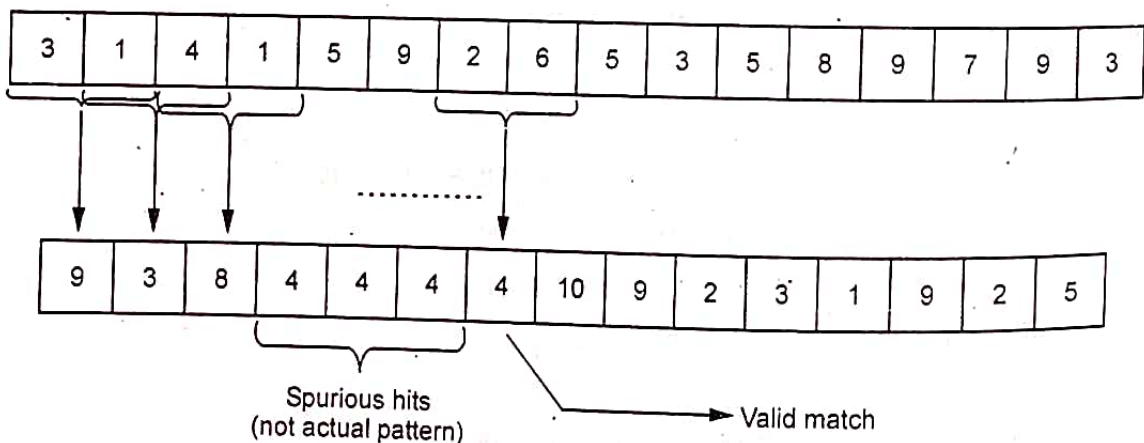
Text	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3
------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Pattern	2	6
---------	---	---

The $26 \bmod 11 = 4$

We will now obtain mod 11 value for each text.

Step 1 :

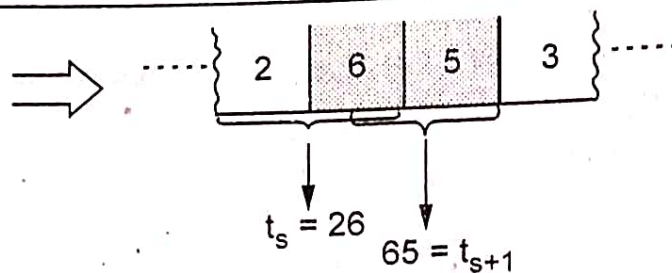


For $m = 2$, consider $t_s = 26$ then we can compute t_{s+1} as follows

$$\begin{aligned} t_{s+1} &= 10(t_s - d_{10}^{m-1} \cdot \text{old high order digit}) + \text{new low order digit} \\ &= 10(26 - 10 \cdot 2) + 5 = 10(26 - 20) + 5 \end{aligned}$$

$$= 60 + 5$$

$$t_{s+1} = 65$$



Thus we can obtain every successive bit in text T.

According to Rabin-Karp method we start matching the pattern against text from beginning itself.

The algorithm is as follows -

3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2	6
---	---

The mod 11 = 9.
Hence shift pattern to right by 1. (As it is not equal to 4)

3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2	6
---	---

Here mod 11 = 3
hence move to right

3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2	6
---	---

The mod 11 = 4.
But pattern is not matching against text.
Hence it is called **spurious hit**.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3

2	6
---	---

The mod 11 = 4,
and pattern is also matching against text.
Hence we declare that at 7th position
we find pattern matching.

Q.13 What are string matching algorithms ? Explain any one algorithm with example.

[SPPU : Dec.-19, Marks 9]

Ans. : Refer Q.8 and Q.11.

END...

SOLVED MODEL QUESTION PAPER (End Sem)

Design and Analysis of Algorithms

B.E. (Computer) Semester - VII [As Per 2019 Pattern]

Time : $2 \frac{1}{2}$ Hours]

[Maximum Marks : 70

N.B. : i) Attempt Q.1 or Q.2, Q.3 or Q.4, Q.5 or Q.6, Q.7 or Q.8.

ii) Neat diagrams must be drawn wherever necessary.

iii) Figures to the right side indicate full marks.

iv) Assume suitable data, if necessary.

Q.1 a) What are the characteristics of greedy method ?

(Refer Q.3 of Chapter - 3)

[8]

b) Explain job-sequencing with deadlines. Solve the following instance : $n = 5$.

$(P_1, P_2, P_3, P_4, P_5) = (20, 15, 10, 5, 1)$

$(d_1, d_2, d_3, d_4, d_5) = (2, 2, 1, 3, 3)$ (Refer Q.8 of Chapter - 3)

[10]

OR

Q.2 a) Explain with example, how to obtain binomial coefficient using dynamic programming ? (Refer Q.17 of Chapter - 3)

[10]

b) Explain 0/1 Knapsack using dynamic programming with an example. (Refer Q.20 of Chapter - 3)

[8]

Q.3 a) What are the constraints that must be satisfied while solving any problem using backtracking ? Explain briefly.

(Refer Q.2 of Chapter - 4)

[5]

b) Explain how backtracking strategy can be used to solve n-queen's problem. Give the pseudocode for the same. Discuss the time complexity for this problem. (Refer Q.9 of Chapter - 4)

[12]

OR

Q.4 a) Explain the algorithm for implementing the branch bound knapsack problem with suitable example.
(Refer Q.26 of Chapter - 4) [10]

b) Explain graph coloring problem with respect to backtracking.
(Refer Q.15 of Chapter - 4) [7]

Q.5 a) What is amortized analysis ? Enlist the techniques used in amortized analysis. (Refer Q.1 and Q.3 of Chapter - 5) [4]

b) What is amortized analysis ? Explain aggregate and accounting techniques with example. (Refer Q.5 of Chapter - 5) [8]

c) Explain the concept of time-space trade-off with suitable example. (Refer Q.6 of Chapter - 5) [6]

OR

Q.6 a) Explain randomized algorithm for quick sort.
(Refer Q.10 of Chapter - 5) [8]

b) What is embedded system ? Explain embedded sorting algorithm. (Refer Q.15 of Chapter - 5) [10]

Q.7 a) Define performance measure of multithreaded algorithms. Write multithreaded algorithm for fibonacci series and explain it with suitable example. (Refer Q.2 of Chapter - 6) [7]

b) Write and explain Rabin-Karp string matching algorithm.
(Refer Q.11 of Chapter - 6) [10]

OR

Q.8 a) Explain multithreaded algorithms. How to analyze multithreaded algorithms ? What is race condition, parallel loops ?
(Refer Q.3 of Chapter - 6) [9]

b) What are string matching algorithms ?
(Refer Q.8 of Chapter - 6) [8]

END...✍

Notes

A series of horizontal lines for writing notes, starting from the top right and extending downwards to the bottom of the page. The lines are evenly spaced and cover most of the page area below the 'Notes' header.



9 789355 852397

Made in India

DECODE SERIES FOR B.E. (COMP) SEM - VII

Compulsory Subjects

1. Design and Analysis of Algorithms
2. Machine Learning
3. Blockchain Technology

Elective Subjects

4. Pervasive Computing
5. Multimedia Techniques
6. Cyber Security and Digital Forensics
7. Object Oriented Modeling and Design
8. Digital Signal Processing
9. Information Retrieval
10. GPU Programming and Architecture
11. Mobile Computing
12. Software Testing and Quality Assurance
13. Compilers

CAUTION : PHOTOCOPYING IS ILLEGAL

DON'T BUY PHOTOCOPIED BOOKS

*This book is protected under **Copyright Act 1999** and sold subject to the condition that the book and any extract thereof **shall be not photocopied** and includes the said condition being imposed on any subsequent purchaser.*

*Any person found selling, stocking or carrying photocopied book may be arrested for indulging in criminal offence of copyright piracy and may be imprisoned for **3 years and also fined a sum of Rs. 2,00,000/- for first offence.***

*Sharing of PDF's, any Drives, Links, Storing in Hard Disks, Pendrive and Circulating on Social Media like Instagram, Telegram, Facebook, Snapchat, Google Drive & Whatsapp etc. also violates the Copyright Laws and will be reported to **Cyber Crime Division.***

Publisher has raided many such offenders. Their Machines were Seized. Criminal case has also been registered against them. Civil Suits are filed for recovering damages.

*Recently, the Supreme Court of India, in **M/s Knit Pro International v. The State of NCT of Delhi** on 20 May 2022, has observed and held that offences under **Section 63** of the Copyright Act, 1957 ("Copyright Act") are **cognizable and non-bailable.***

**Name of informer will be kept highly confidential
and he will be suitably rewarded**

**Call / WhatsApp us on this number
+91 99219 41414**

E-mail: raivindra@technicalpublications.in



